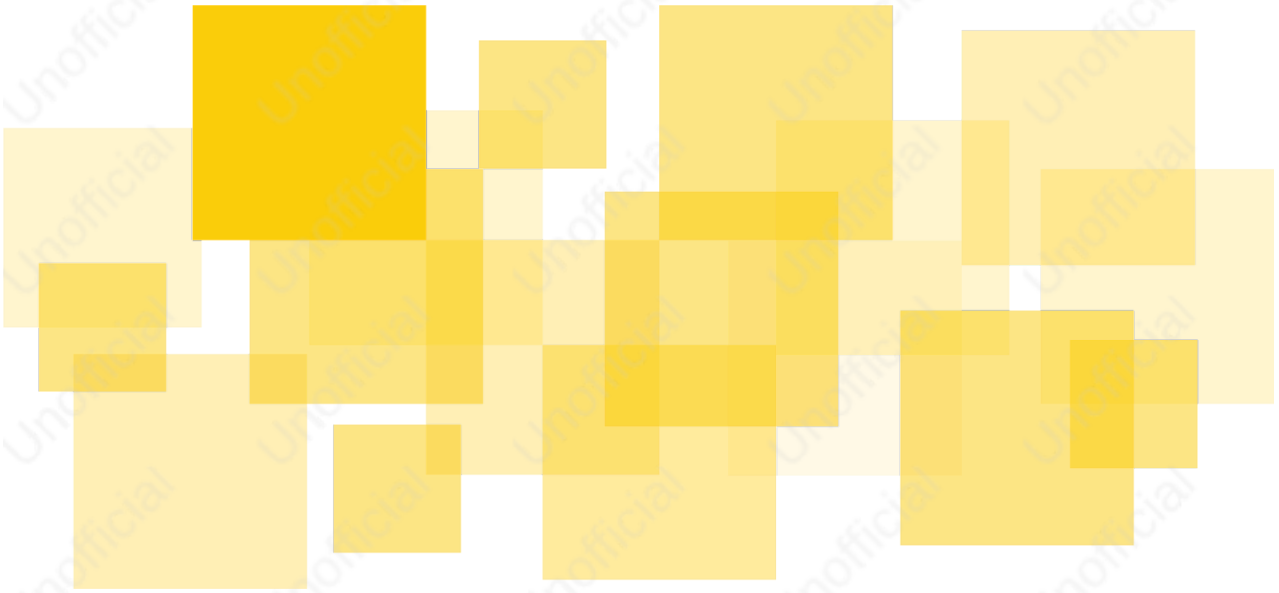
Security Audit Report

OP Supernode and Interop Extension Report Ethereum

Delivered: July 8th, 2026



Unofficial

Table of Contents

- Disclaimer
- Executive Summary
- Scope
- Goal
- Methodology
- Formally-Verified Dafny Model
  - Verification in Dafny
    - Methods
    - Functions and Predicates
    - Lemmas
  - Dafny Source Files
    - `Interop.dfy`
    - `VerifiedDB.dfy`
    - `ChainContainer.dfy`
    - `LogsDB.dfy`
    - `Types.dfy`
    - `Utils.dfy`
  - Simplifying Assumptions
  - Proved Invariants
    - Base Invariants
    - DB Consistency Invariants
    - Cross-Validity of Verified Entries
- Findings
  - Rewind deletes all blocks sealed by backfill
  - Consistency of LogsDBs when initializing from VerifiedDB is not enforced
- Specification-Guided AI Analysis
  - Methodology
  - Results
- Fuzzing
  - Fuzzing Methodology
    - Guiding principles
    - What the harness looks for
  - Fuzzing Harness Architecture
    - Layer 0 — synthetic chain generation (`RandomChain` / `RandomChainManager`)
    - Layer 1 — chain-container read path (`FuzzRandomChainReadPath`)
    - Layer 2 — isolated engine rewind (`FuzzEngineRewind` / `FaultyRandomChain`)
    - Layer 3 — interop verification rounds (`FuzzInteropRound` / `FuzzInteropInvalid`)
    - Determinism and reproduction
  - Fuzzing File Coverage

- Fuzzing Coverage Index
  - `activity/interop/algo.go`
  - `activity/interop/checker.go`
  - `activity/interop/cycle.go`
  - `activity/interop/interop.go`
  - `activity/interop/log_backfill.go`
  - `activity/interop/logdb.go`
  - `activity/interop/reader.go`
  - `activity/interop/types.go`
  - `activity/interop/verification_view.go`
  - `activity/interop/verified_db.go`
  - `chain_container/chain_container.go`
  - `chain_container/engine_controller/engine_controller.go`
  - `chain_container/engine_controller/rewind.go`
  - `chain_container/invalidation.go`
  - `chain_container/super_authority.go`
  - `chain_container/virtual_node/virtual_node.go`
- Appendix: Full AI Analysis Report

Disclaimer

Not addressed by client

This report does not constitute legal or investment advice. You understand and agree that this report relates to new and emerging technologies and that there are significant risks inherent in using such technologies that cannot be completely protected against. While this report has been prepared based on data and information that has been provided by you or is otherwise publicly available, there are likely additional unknown risks that otherwise exist. This report is also not comprehensive in scope, excluding a number of components critical to the correct operation of this system. This report is for informational purposes only and is provided on an "as-is" basis, and you acknowledge and agree that you are making use of this report and the information contained herein at your own risk. The preparers of this report make no representations or warranties of any kind, either express or implied, regarding the information in or the use of this report and shall not be liable to you or any third parties for any acts or omissions undertaken by you or any third parties based on the information contained herein.

Smart contracts are still a nascent software arena, and their deployment and public offering carry substantial risk.

Finally, the possibility of human error in the manual review process is very real, and we recommend seeking multiple independent opinions on any claims that impact a large quantity of funds.

Unofficial

Executive Summary

Not addressed by client

Op Labs engaged Runtime Verification Inc. to conduct a 10-week extension (2 engineers) of the prior Supernode and Interop Filter security audit, building on a 9-week engagement that included design review, code review, and fuzz testing of the cross-chain validation logic, system invariants, and failure modes. The extension started shortly after delivery of the initial audit report, with the objective of strengthening correctness, test coverage, and auditability of the Supernode and Interop Filter ahead of launch.

The extension's primary focus is fully enforcing core invariants of the Dafny formal specification of the Supernode that are currently only partially specified, such as VerifiedDB/LogsDB consistency, cross-validity of VerifiedDB entries, and DenyList correctness, and validating the model against the Go code. Extending the model to cover components not yet modeled, including LogsDB and ChainContainer, was de-prioritized in favor of this work; these components remain abstract, specifying only the expected behavior of each function rather than a concrete implementation. A secondary focus is expanding the existing fuzzing and instrumentation infrastructure to validate these invariants under realistic and adversarial conditions, including partial state transitions, database inconsistencies, and recovery behavior, and to provide a benchmark test suite that can support safe refactoring and a future Rust implementation of the Supernode.

Runtime Verification's process was originally structured into three phases, though in practice the work did not strictly follow this phased breakdown. The team worked continuously with 2 engineers throughout the engagement. The first part of the engagement, around 6 weeks, focused on design and specification alignment: extending the Dafny model, identifying gaps between the model and implementation, recommending refactorings, and expanding fuzzing foundations. The remaining 4 weeks focused mostly on exploring approaches for comparing the model against the Go code and identifying possible violations of the invariants, including an AI-based approach to this comparison.

Scope

Not addressed by client

The scope of this audit was limited to the code contained in the `op-supernode/supernode` directory of the public GitHub repository provided by the Op Labs team, located at `https://github.com/ethereum-optimism/optimism`, with the fixed commit hash `a39ee0e362` on the `develop` branch, which served as the initial revision under review.

The elements within scope are divided into the following core components:

- `op-supernode/supernode` : Contains the implementation of the Supernode, including Interop, ChainContainer, VirtualNode, EngineController, VerifiedDB, LogsDB. Approximately 4,374 lines of Go code across 23 source files, excluding tests.
- `op-supernode/supernode/chain_container` : Contains the interface to an L2 oracle node (e.g. ChainContainer / DenyList logic). Approximately 1,532 lines of Go code across 6 source files, excluding tests.
- `op-supernode/supernode/activity/interop` : Contains the Interop and its validation (e.g. LogsDB / VerifiedDB logic). Approximately 2,505 lines of Go code across 12 source files, excluding tests.

Goal

Not addressed by client

The goals of this extension engagement include:

- Fully enforce core invariants of the Dafny formal specification that were previously only partially specified, including VerifiedDB and LogsDB consistency, cross-validity of VerifiedDB entries, and DenyList correctness
- Extend the Dafny formal specification to cover components not yet modeled (LogsDB, ChainContainer/DenyList), where feasible within the engagement's priorities; where not fully modeled, these components are left abstract, specifying expected behavior of each function without a concrete implementation
- Identify gaps between the model and implementation by validating the model against the Go code, including exploring AI-based approaches to systematically compare model and implementation and surface possible invariant violations, and recommend refactorings where the two diverge
- Validate core mechanisms, database consistency under failure, and cross-chain invariant enforcement through formal verification where provable, and manual/system-level review where not
- Expand fuzzing and instrumentation to align test coverage with the modeled invariants, including edge-case tests (e.g. expired messages, invalid state transitions) and fault injection (partial state updates, database failures, ingestion errors)
- Provide continuous validation during implementation changes to catch regressions or architectural drift early
- Establish a reusable specification and fuzz-suite to guide future refactors and alternative implementations (e.g. a Rust Supernode), including differential testing between implementations

The extension builds on the initial audit by deepening formal assurance around invariants and failure modes identified but not fully explored previously, particularly database consistency and cross-component boundaries. It also aims to reduce the cost of future development by making assumptions explicit, catching regressions earlier via CI-integrated fuzzing, and providing a specification layer to guide both human and LLM-assisted development.

Methodology

Not addressed by client

This extension builds directly on the design review, formal model, and property specification produced during the initial engagement. Rather than repeating a full design review, this engagement focused on fully enforcing core invariants in the existing Dafny model and on validating that model against the Go implementation. The engagement was originally scoped into three phases: Design and Specification Alignment, Implementation Support and Continuous Validation, and Final Validation. In practice, the work did not strictly follow this phased breakdown, the team worked continuously with 2 engineers throughout, splitting effort roughly as described below.

Model Extension & Invariant Verification (6 weeks)

- Formalize additional invariants identified as gaps in the initial engagement, including VerifiedDB/LogsDB consistency, cross-validity of VerifiedDB entries, and DenyList correctness, and prove them against the Dafny model.
- Extend the fuzzing and instrumentation foundations to align coverage with the newly formalized invariants.
- De-prioritize extending the model to previously unmodeled components, namely LogsDB and ChainContainer, in favor of the invariant work above. These components remain abstract in the model, specifying only the expected behavior of each function rather than a concrete implementation.

Model-to-Implementation Comparison (4 weeks)

- Explore approaches, including an AI-based approach, for systematically comparing the Dafny model against the Go implementation to identify mismatches between specified and actual behavior.
- Use findings from this comparison to identify possible violations of the invariants established in the first part of the engagement.
- Flag divergences between the model and implementation for follow-up, where alignment would materially reduce risk.

Throughout the engagement, code review, modeling, and fuzzing informed one another on an ongoing basis: mismatches surfaced by the model guided fuzzing targets, and fuzzing results in turn helped validate or refine the model itself.

Formally-Verified Dafny Model

Not addressed by client

During the course of the extension, we have built an abstract model of the Supernode in Dafny, a programming language with built-in formal verification features, and then formally verified that certain important invariants of the Supernode hold in this model. The current version of the model can be found at https://github.com/runtimeverification/_audits_Ethereum-optimism_optimism_interopv2/tree/rv/dafny-models-extension/op-supernode/dafny-models.

Verification in Dafny

Methods

Methods in Dafny are procedures that can mutate the state of the program. They can be defined at the top level of the program or belong to a class. In addition to input parameters and return values, a method signature can also include the following types of clauses:

- `requires` clauses encode the method's *preconditions*: expressions over the input parameters and the program state that must be true at the start of the method. If Dafny cannot prove that a precondition holds when a method is called, verification fails.
- `ensures` clauses encode the method's *postconditions*: expressions over the input parameters, return values and program state that must be true at the end of the method. When verifying a method, Dafny assumes that the preconditions hold and tries to prove that, under this assumption, the body of the method guarantees that the postconditions hold. If it cannot, verification fails. `ensures` clauses can also use the keyword `old`, which can be used to refer to the value of a state variable at the *start* of the method. This can be useful to express postconditions that compare the initial and final states of a variable that the method modifies.
- `modifies` clauses list all variables or sets of variables that the method modifies. If the body of a method can modify a variable that is not included in the `modifies` clause (including by calling another method that modifies that variable), verification fails.

Method bodies might include `assert` statements to indicate intermediate expressions to be verified, which can help guide the verifier when proving trickier properties. Verification fails if Dafny is unable to prove that the `assert` holds. `assume` statements can also be used to specify facts to be assumed without proof. Dafny automatically adds these to the context when verifying the rest of the method body.

When a method body includes a loop, Dafny might require loop invariants to complete verification. These are specified with the `invariant` keyword, and placed directly before the body of the loop. A loop invariant must be true when the loop is first entered, and Dafny verifies that it is preserved by the loop body. That way, it can guarantee that the invariant is necessarily true when exiting the loop.

Dafny also allows methods to be left completely abstract, without a body. Although code with abstract methods cannot be compiled and executed, it can still be verified. `ensures` clauses of an abstract method need to be annotated with the `{:axiom}` attribute to indicate that we are assuming that they hold without proof. When an abstract method is called by another method, Dafny uses its preconditions and postconditions when verifying the caller, just like any other method.

Functions and Predicates

Unlike methods, functions in Dafny are pure, meaning that they can't mutate state. However, this also means that, unlike methods, they can be used in `requires` and `ensures` clauses. Predicates are simply functions that return a boolean.

Since functions and predicates can't mutate state, they don't have a `modifies` clause, but they can have `requires` and `ensures` clauses. Unlike methods, they also require a `reads` clause listing all variables or sets of variables that the function depends on (other than its input parameters). This is so Dafny knows what functions are affected when the value of a variable changes.

Additionally, functions and predicates can be annotated with the following keywords:

- `ghost` denotes a function or predicate that is not compiled and cannot be used in compiled code (such as the body of a method). However, they can be used in specifications, such as `requires` and `ensures` clauses, as well as in the body of other `ghost` functions or predicates. In other words, they only exist at the formal verification stage.
- `opaque` prevents the body of the function or predicate from being visible to Dafny at the point where it is used. Normally, Dafny can use a function's definition at the point where it's called to try to prove other properties, but for performance reasons sometimes it is useful to make it `opaque` in order to not overwhelm Dafny's reasoning engine. The definition of an `opaque` function or predicate can be placed back into the context by using the `reveal` keyword.
- `twostate` denotes that a function or predicate is defined over the value of certain variables at two different points in the program (using the `old` keyword). This can be useful to define properties about how certain variables have changed. The "current" state is the point where the function or predicate is called, and by default the "old" state is the beginning of the method where it's called. However, by defining a `label` and using the notation `<function name>@<label>` , a `twostate` function can be called using the point where the label was defined as the "old" state.

Lemmas

Sometimes Dafny needs help proving that certain properties hold. In that case, the user can define a helper lemma. Lemmas are defined similarly to methods, with `requires` and `ensures` clauses. However, they don't change the state or perform any concrete computation. Instead, they only serve to prove that the preconditions logically imply the postconditions. The body of the lemma is verified by Dafny similarly to a method, and then the lemma can be called in other methods in order to help with the proofs.

Like functions and predicates, lemmas can also be annotated with the `twostate` keyword. This is useful, for example, when proving that if a certain property holds before a method is called, it still holds after the method returns (without having to add it to the method's preconditions and postconditions explicitly).

Dafny Source Files

The Dafny model of the Supernode consists of the following source files.

`Interop.dfy`

The main component of the model, defining an `Interop` class that contains the bulk of the Supernode logic. The main function is `ProgressAndRecord` , which implements the core of the cross-validation loop. In addition to the methods implementing the cross-validation process, it defines several predicates encoding the invariants of the Supernode (see below) as well as other intermediate properties used as preconditions and postconditions of various methods. It also includes several lemmas, used mostly to prove preservation of various properties by method calls.

Some of the methods in the `Interop` class are left abstract, with only their preconditions and postconditions specified:

- `VerifyCycles` , representing the cross-validation step that checks that executing messages at the same timestamp don't form a cycle.
- `ResolveFrontierVerificationView` , which builds the frontier verification view by fetching data from the L2 chain.
- `ComputeL1Inclusion` , which computes the highest L1 block among the blocks the L2 heads were derived from.
- `ProcessBlock` , which seals a block's log into the `LogsDB` for the corresponding L2 chain.
- `CheckL1Consistent` , representing the check that the L1 blocks from which the L2 heads were derived, as well as the current last verified L1 origin in the `VerifiedDB` , are all part of the same canonical chain.
- `RefreshCurrentL1OnWait` , representing the update of the `currentL1` variable when there is no progress in the cross-validation (correctness of the `currentL1` is not in scope for the verification).

`VerifiedDB.dfy`

A simplified model of the `VerifiedDB` , containing a map of timestamps to `VerifiedResult` s at those timestamps. It implements methods that update the database (`Commit` , `Rewind` , `SetPendingTransition` , `ClearPendingTransition`) as well as functions that

read the state (`Get` , `Has` , `LastTimestamp` , `GetPendingTransition`). The logic is simplified by using `requires` clauses to enforce function preconditions rather than checking them at runtime and returning an error. A predicate `Valid()` represents the internal invariants of the `VerifiedDB` :

- The verified timestamps are sequential with no gaps.
- The timestamp of each `VerifiedResult` matches its key.
- Block numbers for each L2 chain are monotonically increasing.
- The `lastTimestamp` field stores the highest timestamp saved to the database.

These invariants are assumed as preconditions and enforced as postconditions for every method, thus verifying that they are preserved whenever the database is updated.

`ChainContainer.dfy`

An abstracted model of the `ChainContainer` . All methods are abstract, with only their preconditions and postconditions specified, so this serves as a specification of how we assume the `ChainContainer` to behave. The methods implemented can be split into two groups:

- Methods that update the state of the `ChainContainer` and the L2 chain (`PruneDeniedAtOrAfterTimestamp` , `RewindEngine` , `InvalidateBlock`): Since this was not the focus of the modeling and verification work (which was more focused on the internal state of the Supernode) these methods are left underspecified beyond just a `modifies this` clause indicating that they mutate the `ChainContainer` state.
- Methods that query the chain state (`OptimisticAt` , `FetchReceipts`): Although these methods are technically read-only, they include a `modifies this` clause to model the fact that the chain can spontaneously change between calls due to reorgs. In practice, this is logically equivalent to assuming these methods can arbitrarily mutate the chain state.

In addition to these methods, the `ChainContainer` class also includes three functions that are independent of the current chain state (to make this explicit, they include a `reads {}` clause, indicating that they always return the same result regardless of the current state):

- The first function is `BlockTime` , which simply returns the block time (the time between consecutive blocks) configured for that chain.
- The other two functions are `BlockInfo` and `BlockLogs` . These functions are special because they are meant to return information about blocks *regardless of whether or not those blocks are included in the current canonical chain*. In other words, given a block ID (block number and hash), these functions will return the block's data even if that block is no longer in the chain due to a reorg. Because they are declared as `ghost` functions, they cannot be called from method bodies, and instead are meant to be used only in specifications. They are used in the model to ensure consistency between queries to the chain. For example, they allow us to specify and verify that the logs stored in the `LogsDB` actually correspond to the block's logs fetched from the L2 chain.

`LogsDB.dfy`

Similarly to the case for `ChainContainer` , this is an abstract model of the `LogsDB` where all methods and functions lack bodies and are solely specified in terms of their preconditions and postconditions. Since many of the invariants of the Supernode proved in the model (see below) relate to the `LogsDB` s, the methods of this class are specified in more detail than for the `ChainContainer` class. Examples of important `LogsDB` behaviors specified via its preconditions and postconditions are:

- `FirstSealedBlock` and `LastSealedBlock` return respectively the first and last blocks in the database, meaning that `FindSealedBlock` returns `None` (Dafny equivalent to `nil`) for every block number outside of this range. If they return `None` , the database is empty.
- `FindSealedBlock` either returns `None` (if the block is not in the database) or a block with the same block number given as input.

- `Rewind` can only be called with a `BlockID` that is present in the database, and in that case that block becomes the new `LastSealedBlock`. Additionally, all blocks with a lower block number remain in the database and their information doesn't change.
- `Clear` makes the database empty (`FirstSealedBlock`, `LatestSealedBlock` and `FindSealedBlock` all return `None`).

Note that the `LogsDB` class in the model doesn't include explicit `AddLog` and `SealBlock` methods to insert new blocks into the database. This functionality is instead modeled in the abstract `ProcessBlock` class in `Interop.dfy`. The fact that these methods are not directly specified doesn't affect verification, since the postconditions of `ProcessBlock` already describe how the `LogsDB` is expected to change when a new block is added.

`Types.dfy`

This file contains Dafny type definitions for various data structures used in the model (most of them analogous to data structures from the Go code). These include, for example: `BlockID`, `VerifiedResult`, `PendingTransition`, `RewindPlan`, `RoundObservation`, `StepOutput`, `ExecutingMessage`, `BlockLogs`. It also defines predicates that specify what is a "valid" instance of one of these structures, such as `ValidPendingTransition`, `ValidRewindPlan` and `ValidStepOutput`. Every instance of these types created in the model should satisfy the corresponding predicate, and is verified as such.

`Utils.dfy`

This file contains generic functions, predicates and lemmas that are used by the `VerifiedDB` model. This includes functions for obtaining the max element of a set or key of a map, a predicate defining what it means for a map to be sequential without gaps, and lemmas proving properties related to those.

Simplifying Assumptions

- It's important to note that the verification depends on the assumptions of how the `LogsDB` and `ChainContainer` behave specified by their abstract models. The specifications were designed to cover the expected range of behaviors for these components, but exceptional circumstances (for example, disk corruption of the `LogsDB`) could fall outside of their scope.
- For simplicity, in order to focus on the core logic of the components, the models of the `VerifiedDB` and `LogsDB`s use `requires` clauses to specify their function preconditions and verify that they hold when these functions are called. These include conditions that in the Go implementation are checked at runtime and could return an error if the functions are called incorrectly. The model assumes that as long as the function preconditions are satisfied, the functions will return correctly. As mentioned above, this assumption excludes exceptional circumstances like corruption of the databases on disk. On the other hand, the model of the `ChainContainer` assumes that its functions may always fail and return an error, since they depend on network communication, which may be unreliable.
- There are three points in the model where `assume` statements are used to have Dafny assume facts without proof. These are consistency conditions between the block at the current timestamp `ts` and the latest sealed block in the `LogsDB` at the previous timestamp `ts - 1`, and should be guaranteed by the self-consistency of the L2 chain as long as there hasn't been a reorg. Since `ObserveRound()` checks for reorgs and triggers a rewind if one is detected, it is safe to assume that these consistency properties hold in the case where no rewind is triggered. The three `assume` statements are the following:
  - The first two are in the `CheckChainsReady` method, where we assume the following two facts: (a) the block number of the current block at `ts` is either equal to or 1 higher than the latest block at `ts - 1`, and (b) if the two block numbers are equal, then the two blocks have the same hash (are the same block).
  - The third is in the `PersistFrontierLogs` method, where we assume that, if we are adding a new block to the `LogsDB`, then its parent hash matches the hash of the latest sealed block (its predecessor).
- The model does not cover the cold-start initialization and backfill, which may populate the `LogsDB`s with past blocks that new cross-validated executing messages might depend on. Instead, it assumes that cross-validation starts from empty `VerifiedDB` and `LogsDB`s. This should not significantly affect the semantics of the model, except in the case where the Supernode has to

rewind past the initial verified timestamp (see Rewind deletes all blocks sealed by backfill for a scenario where this might make a difference).

Proved Invariants

The following is the core set of invariants that are proved over the Dafny model of the Supernode. To verify these invariants, we add them as both preconditions and postconditions of the `ProgressAndRecord` method. Therefore, passing verification guarantees that, as long as they hold when `ProgressAndRecord` is called, they will continue to hold afterwards. By adding them as postconditions in the constructor, we also prove that they hold in the initial state where all DBs are empty.

Base Invariants

These invariants are captured in the predicate `Valid()`, and represent basic internal consistency properties of the Supernode, such as:

- The `VerifiedDB` invariants hold (as described in the `VerifiedDB.dfy` section above).
- The `VerifiedDB` starts at the activation timestamp.
- If there is a `PendingTransition` stored in the `VerifiedDB`, it is valid according to the `ValidPendingTransition` predicate defined in `Types.dfy` (for example, the `PendingTransition` only has a `RewindPlan` if the decision is `Rewind` and only has invalid heads if the decision is `Invalidate`).
- The data in the `VerifiedDB` and `LogsDB`s is consistent with the data obtained from the chain (for example, the logs recorded in the `LogsDB` for a block are correct, and the L2 heads in the `VerifiedDB` for a timestamp are the highest blocks for each chain up to that timestamp). This invariant should hold even for blocks that may no longer be canonical due to a reorg (see the `BlockInfo` and `BlockLogs` functions in the `ChainContainter.dfy` section above).
- All maps that should have an entry for each L2 chain have the same number of chains (`LogsDB`s, `ChainContainer`s, L2 heads in a record of the `VerifiedDB`, etc.).
- No two chains use the same `LogsDB` (necessary to prove that there are no conflicts).

DB Consistency Invariants

These invariants are captured in the predicate `PendingTransitionIsConsistent()`, and specify the properties necessary to ensure that, after a `PendingTransition` is successfully applied, the `VerifiedDB` and `LogsDB`s are consistent with each other. This can be split into two parts:

- If there is no `PendingTransition` currently stored in the `VerifiedDB`, then the `VerifiedDB` and `LogsDB`s are in sync, meaning that every block in the `VerifiedDB` is also in the `LogsDB` for the corresponding chain.
- Otherwise, the `PendingTransition` is such that, *after* it's applied successfully, the `VerifiedDB` and `LogsDB`s will be in sync. For example, the `VerifiedDB` can be behind the `LogsDB`s by one timestamp if the `PendingTransition` is `Rewind` or `Advance`, so that it will either rewind the `LogsDB`s or advance the `VerifiedDB`, and thus return them to being in sync.

These invariants essentially guarantee state recovery after a failed attempt to apply a `PendingTransition`. In other words, if there is an error during the state update, the Supernode is left in a state that will return to synchrony once the `PendingTransition` is re-applied successfully in a later iteration.

Cross-Validity of Verified Entries

These invariants guarantee that all entries in the `VerifiedDB` are cross-valid. Specifically, for every L2 head in the `VerifiedDB`, the following are true for each of its executing messages:

- Both it and its initiating message are at least a full block after the activation timestamp (by the block time of their respective chains).
- Its initiating message is in the past and not expired.

- Its initiating message is present in the `LogsDB` for the corresponding L2 chain.

Note that this proof does not include the same-timestamp cycle check, which is abstracted away in the Dafny model.

Additionally, if there is a `PendingTransition` and it's an `Advance`, the same properties apply to every executing message in the blocks that it's inserting. This is to ensure that the invariants will hold once the `PendingTransition` is successfully applied. The invariants over the `VerifiedDB` are captured by the predicate `AllVerifiedCrossValid`, while the same invariants over the `PendingTransition` are captured by the predicate `TransitionIsCrossValid`.

Findings

Not addressed by client

This section contains all issues identified during the audit that could lead to unintended behavior, security vulnerabilities, or failure to enforce the protocol's intended logic. Each issue is documented with a description, potential impact, and recommended remediation steps.

Unofficial

Rewind deletes all blocks sealed by backfill

Not addressed by client

The updated version of the Supernode has two different initialization paths:

1. **Resume from `VerifiedDB`** : This path is taken when the Supernode is being restarted after a previous interruption, and there are already verified entries present in the `VerifiedDB` from a previous execution. In this case, the `VerifiedDB` is taken as a source of truth, and the `LogsDB` s are trusted to be consistent with the `VerifiedDB` . `verificationStartTimestamp` (the timestamp from which we start performing cross-validation) is set to the next timestamp after the last timestamp present in the `VerifiedDB` .
2. **Cold-start**: This path is taken when the `VerifiedDB` is empty when the Supernode is initialized. In this case, the Supernode queries the node for each L2 chain for the first local-safe block recorded in its `SafeDB` and sets `verificationStartTimestamp` to the highest timestamp among these blocks (or to `activationTimestamp` if it is higher than all of them). It then backfills each `LogsDB` s with all past blocks after `activationTimestamp` that are within the message expiry window (in other words, these are all the blocks whose messages can still be referenced by executing messages at `verificationStartTimestamp`).

Therefore, in the cold-start initialization path, the `VerifiedDB` will contain entries for timestamps from `verificationStartTimestamp` onwards, while the `LogsDBs` might contain additional past blocks before this timestamp. However, if later there is a rewind to a timestamp before `verificationStartTimestamp` (thus clearing the `VerifiedDB`), this will cause the `LogsDB` s to be cleared as well, deleting all of the past blocks added by backfill. This happens because `buildRewindPlan` uses the `VerifiedDB` entry at the rewind target timestamp to calculate the `TargetHeads` :

runtimeverification/_audits_Ethereum-optimism_optimism_interopv2/op-supernode/supernode/activity/interop/interop.go

Line 955 to 959 in a5cfcc2

```
955  This file is no longer available — it may be on a deleted branch.
```

As a result, if `rewindTargetTS == verificationStartTimestamp - 1` , the `VerifiedDB` will not have an entry for this timestamp, and `TargetHeads` will be `nil` . Then, in `applyRewindPlan` , if `TargetHeads == nil` the `LogsDB` s are cleared completely:

runtimeverification/_audits_Ethereum-optimism_optimism_interopv2/op-supernode/supernode/activity/interop/interop.go

Line 1054 to 1060 in a5cfcc2

```
1054  This file is no longer available — it may be on a deleted branch.
```

After this, there is no way to reinsert the backfill blocks in the `LogsDB` s. This may cause problems if in the post-reorg chain there are executing messages at `verificationStartTimestamp` or later that reference initiating messages in these past blocks. The existence of these initiating messages will not be able to be verified, and therefore cross-validation will incorrectly reject these valid executing messages.

Consistency of LogsDBs when initializing from VerifiedDB is not enforced

Not addressed by client

When initializing the Supernode from a preexisting `VerifiedDB`, there is no validation that the contents of the `LogsDB`s are consistent with the `VerifiedDB` entries. This should be true if both the `VerifiedDB` and `LogsDB`s remain as they are from a previous execution, but if the databases have been corrupted for any reason the Supernode will not detect this upon initialization, which might cause problems in future execution (for example, if a block that should be in the `LogsDB` is missing, it might produce an error when trying to insert its successor, or it might cause cross-validation of an executing message to fail because its initiating message can't be found).

Specification-Guided AI Analysis

Not addressed by client

As a way of using the formally-verified Dafny model to validate the Go code, we explored a systematic AI-based approach to analyze the code based on the specification provided by the model.

Methodology

The AI analysis made use of existing infrastructure developed for RV's AI audit assistant, which is used to identify function preconditions and postconditions over the code being audited. In this case, however, the objective is to instead check the code against the preconditions and postconditions specified in the Dafny model. The analysis consisted of the following stages:

1. Constructing a callgraph for the Go code using Trail of Bits' Trailmark tool, containing information on the caller-callee relationships between all functions.
2. Defining a map between the methods in the Dafny model and the Go function it corresponds to.
3. Traversing the callgraph and performing for each function an analysis by an AI agent to validate whether the Go implementation satisfies the preconditions and postconditions specified in the Dafny model.
4. Creating a report documenting the results of the analysis for each function and detailing any detected violations.

The analysis in step 3 was performed in a modular way, with each function being examined in isolation by an independent agent instance. The analysis for a given function was split into:

- **Precondition analysis:** Check all callers of the function (according to the generated callgraph), and ensure that we can guarantee at the callsite that the function's preconditions are satisfied, based on the caller's preconditions and the specifications of any other functions called previously.
- **Postcondition analysis:** Analyze the body of the function to check that, assuming the function's preconditions are satisfied at entry, the implementation ensures the postconditions will be satisfied on exit.

When analyzing the body of a function, whether to determine if it satisfies its own postconditions or if it satisfies the preconditions of one of its callees, the result might also depend on the behavior of other functions that it calls. Therefore, we also provide as part of the context the Dafny specification of all of the function's callees, including their `requires`, `ensures` and `modifies` clauses. This makes the analysis modular and prevents having to include the bodies of these functions in the context. Since we also analyze the callee itself for compliance with the specification, this analysis is sound.

Results

The full report produced by the analysis can be found in Appendix: Full AI Analysis Report. Each precondition is classified as *Satisfied*, *Violated* or *Uncertain*, along with a description of potential violation scenarios for Violated and Uncertain results. The functions with reported Violated results are the following:

- `processBlockLogs`: The Dafny model includes the postcondition that, when a block is sealed to a `LogsDB`, the data for all other blocks remain the same in the database. This is not strictly true in the Go code, because when adding the first block in an empty database, a virtual parent is also added with the preceding block number. This is an implementation detail that is a result of how the database is implemented, since opening a block requires the existence of a parent block. However, the model omits this for simplicity in order to more explicitly capture the underlying logic of the `LogsDB`.
- `buildPendingTransition`: Certain postcondition violations are reported for this function in the `Rewind` case, due to the new initialization procedure, which is not covered in the model. The model assumes that cross-validation starts at the activation timestamp, but the new cold-start initialization and backfill procedure in the code means that the first verified timestamp might be later than the activation timestamp. This means that a rewind might leave the `VerifiedDB` and `LogsDB` empty before the activation timestamp, which is not anticipated by the postconditions.

- `verifyInteropMessages` : This violation is related to a piece of code in the Go implementation of the function that should be unreachable. The relevant case only happens if the expected block is not present in the `frontierVerificationView` , which shouldn't be possible in the current implementation.
- `observeRound` : This violation results from a slight mismatch between the definition of `RoundObservation` in the model and in the Go code. In the model, the flag `l1Consistent` indicates whether the latest L1 head in the `VerifiedDB` is in the same canonical chain as the L1 origins of the newly-observed L2 heads. If it is `false` , this indicates that the L1 has suffered a reorg and the Supernode must be rewound back. In the latest version of the Go code, however, `L1Consistent` instead indicates whether the L1 origins of the newly-observed L2 heads are all in the same canonical chain as each other, while `L1NeedsRewind` is the flag that indicates the same as `l1Consistent` in the model. Because of this mismatch, the analysis flags a violation because it expects `L1Consistent` to be equivalent to `l1Consistent` .

The above review suggests that the violations are largely benign and result from expected mismatches between the Go code and the model. The most significant divergence is probably the one affecting `buildPendingTransition` , which could theoretically lead to issues such as the one described in Rewind deletes all blocks sealed by backfill.

The Uncertain results also seem benign. Some of the sources of uncertainty relate to the same mismatches between the Go code and Dafny models as described above, as well as other differences such as the fact that the model abstracts away some error paths (as described in Formally-Verified Dafny Model). Other Uncertain results are due to functions in the Go code that lack specifications in the Dafny model (due to either being abstracted away or being inlined in other functions), making it hard for the analysis to determine that they don't make changes to relevant components and do preserve invariants. Details can be found in the report.

Fuzzing

Not addressed by client

This section presents an overview of the fuzzing methodology, a description of the fuzzing targets and random generators that compose the implementation, as well as a coverage report.

Fuzzing Methodology

Not addressed by client

We took lessons learned from the development of the harness in the previous engagement and wrote a new harness that replaces less of the supernode's functionality than the previous harness. Before, we made an implementation of the `ChainContainer` that the Interop uses to retrieve L2 information from the nodes. However, below the `ChainContainer` lie `L2Provider` and `EngineController` which act as more minimal layers over the L2 nodes that the Interop's actual chain container implementation uses to manage node state. Lowering our implementation to those interfaces exposed more of the Interop's functionality, notably the rewinding logic.

Guiding principles

Exercise as much production code as possible.
The harness wires the genuine `Interop` verifier, the L1-consistency checker, `logsDB` / `verifiedDB` stores in a temp directory, and the `EngineController` rewind path. The only substituted component is the data source behind the virtual-node and engine interfaces: a generated in-memory chain (`RandomChain`) stands in for a live `op-node` / execution client.

Keeping the production code untouched and avoiding making any modifications for the sake of the testing harness is paramount to ensure that we don't change any behavior of the code we're testing. However, to employ fault-injections into the operations of the `verifiedDB` , we made a very minor structural change which allowed us to set up an `Interop` that holds a wrapper struct containing the verifiedDB that has the exact same interface. When this struct is un-armed and wont produce any faults then the behavior that's expected by the Interop should remain exactly the same.

Valid-path first, then adversarial.
We built the harness in a strict order: first generate only valid, internally-consistent chains and prove the verifier accepts them cleanly and reaches the safe head; only then introduce corruption, divergence, and faults. This ordering matters because a fuzzer that starts from malformed input mostly measures how early a parser/system rejects garbage. By establishing a large valid input space first, every subsequent planted violation is a *single*, well-understood deviation from a known-good baseline, so a failure isolates the defect rather than the surrounding noise.

A formal-model oracle, not hand-rolled assertions.
The correctness oracle is a set of Go predicate checkers (`dafny_checks.go`) transliterated conjunct-for-conjunct from the project's Dafny model of the interop state machine (`Interop.dfy` and siblings). Each checker mirrors a named model predicate — `Valid()` , `DBsInSync` , `AllDBsInSyncUpTo` , `ValidPendingTransition` , and so on — down to the individual conjunct labels, and reports violations against the Interop after each verification round. Using the model as the oracle means the harness checks the invariants that were discovered during the development of the formal Dafny model. (The Dafny sources themselves were dropped from the fuzzing branch once the checkers were validated against them; the Go checkers are retained as the executable oracle.)

Deterministic, reproducible runs.
Every run is a pure function of the fuzz input bytes. The input seeds an FNV-hashed PRNG (`NewRandomChainManager`), which drives chain topology, block contents, and violation selection. The same input always reproduces the same chain set and the same outcome, so any failure the fuzzer surfaces is replayable from its corpus entry with no external state.

Assert continuously, not just at the end.
The model invariants are asserted *every round*, not only at termination. On the valid path this means the verifier must remain model-valid at each step from the first verifiable slot to the safe head. On the adversarial path, after the verifier rejects a planted violation the state must still be model-valid — rejection is required to leave the machine in a consistent state, not merely to return an error.

What the harness looks for

- **Safety violations** — the verifier accepting an inconsistent cross-chain state, or advancing on data it should reject.

- **Invariant breaks** — any live divergence from the Dafny model predicates.
- **Crashes / panics** — on both valid and malformed inputs, across the data reads and the rewind path.
- **Incorrect rejection** — a planted violation producing the *wrong* rejection (e.g. a silent advance where a `DecisionInvalidate`, `DecisionWait`, `DecisionRewind`, or a hard `ErrHistoryUnavailable` is required), or leaving inconsistent state behind.
- **Non-idempotent / non-recoverable recovery paths** — the engine rewind failing to converge, double-inserting, or reporting success without reaching the target.

Fuzzing Harness Architecture

Not addressed by client

The harness is organised in layers, from the synthetic data generator at the bottom to the round-level verifier driver at the top.

Layer 0 — synthetic chain generation (`RandomChain` / `RandomChainManager`)

`RandomChainManager` turns fuzz-input bytes into a set of valid, internally-consistent chains sharing one canonical L1. `Generate()` picks a fixed-shape topology (2–4 chains, even L2 depths of 4–16 blocks) so that every chain's SafeDB window aligns and the verification frontier is well-defined; the fuzz input varies block *contents*, not topology. For each chain it produces:

- an L2 block sequence, each block carrying a planted initiating log (index 0) and, after wiring, one executing message referencing an earlier verifiable block on *another* chain (`wireExecutingMessages`);
- a sparse, ascending **SafeDB** (`SafeHeadEntry` rows) mapping L2 safe heads to the L1 block at which they became safe;
- the head labels (unsafe / safe / finalized L2, current / finalized L1), chosen so the finalized head sits at or below safe and below the current L1.

`RandomChain` implements the two production interfaces the verifier depends on: the `virtual_node.VirtualNode` read interface (`SafeHeadAtL1` , `L1AtSafeHead` , `FirstSafeHeadEntry` , `SyncStatus` , and the `safeDB Lookup` binary search) and the `engine_controller` `l2Provider` set (`L2BlockRefByLabel/ByNumber` , `OutputV0AtBlock*` , `PayloadBy*` , `FetchReceipts` , `ForkchoiceUpdate` , `NewPayload`). It also carries a minimal forkchoice engine: `ForkchoiceUpdate` records the requested heads so the rewind path can be driven end-to-end. `RandomChainManager` additionally exposes a `RandomL1Source` feeding the Phase-1 L1-consistency checker from the same canonical L1, so L1 reorgs modelled in the generator propagate to the checker.

Layer 1 — chain-container read path (`FuzzRandomChainReadPath`)

The lowest fuzz target drives the `simpleChainContainer` ingestion — the surface the interop verifier reads through — and asserts the model answers consistently and never panics. For every generated block it round-trips `OutputV0AtBlockNumber` , `BlockNumberToTimestamp` / `TimestampToBlockNumber` , `FetchReceipts` (checking every log decodes via the production `DecodeExecutingMessageLog`), and the safe-head methods (`FirstSafeHeadTimestamp` , `ELFinalizedHead` , `OptimisticAt`). This layer establishes that the generated data is a faithful stand-in before any higher layer trusts it.

Layer 2 — isolated engine rewind (`FuzzEngineRewind` / `FaultyRandomChain`)

This target isolates the `EngineController.RewindToTimestamp` state machine, which the interop invalidate/rewind path relies on. `FaultyRandomChain` wraps a `RandomChain` and models an execution layer whose *reported* head can diverge from what forkchoice requested — the divergence the rewind must survive.

`assertRewindInvariants` encodes the correct post-fix behaviour as invariants I1–I4. The idempotency invariants (I2/I3) are **known-broken on the pinned audit commit** — the harness reproduces issue #20929. Those specific assertions are intentionally commented out so the target runs in *exploration mode* — continuing past the known bug to reach downstream states — with the trigger seeds (`trigger-i2` , `i3-stuck`) and the reproduction step (uncomment the assertion) documented inline.

Layer 3 — interop verification rounds (`FuzzInteropRound` / `FuzzInteropInvalid`)

The top layer wires a full `Interop` verifier over a generated chain set (`buildInterop`): real `logsDB` s and a real `verifiedDB` in a temp directory, the real L1 consistency checker, a real (empty) deny list per chain matching production's `NewChainContainer` , started and positioned at the first verifiable slot.

`FuzzInteropRound` drives the valid path.
It loops `progressAndRecord()` to the safe head, asserting after *every* round that the Dafny invariants hold (`AssertInvariants`) and, on a ready round, that the live observation is relationally consistent with both the `verifiedDB` and the sealed logs. Valid data must verify cleanly and reach exactly the safe head's slot.

`FuzzInteropInvalid` drives the adversarial path.
`RandomChainManager.BreakOne()` plants exactly one violation, drawn uniformly across the reachable violation classes, and returns a `Plan` describing how to drive to the trigger and what rejection to expect:

Class	Injection	Required verdict
`RejectInvalidHead`	corrupt one executing message (checksum, dangling index, wrong origin, or ancient/expired timestamp)	`DecisionInvalidate` , target chain in `InvalidHeads` , then apply the invalidation (freeze VNs, deny-list, engine rewind)
`RejectWait`	diverge one chain's frontier L1 head from canonical	`DecisionWait`
`RejectHistoryUnavailable`	raise a chain's earliest SafeDB coverage above the frozen start (front gap)	hard `ErrHistoryUnavailable`
`RejectRewind`	reorg the canonical L1 below a recorded inclusion (probe at fork=1, or realistic at fork=finalized+1)	`DecisionRewind` , `verifiedDB` rewinds below its prior last timestamp

Static classes drive round-by-round until the frontier reaches the planted block, then assert the verdict. The temporal (reorg) class drives until a recorded L1 inclusion reaches the fire threshold with a ready round still remaining, fires the reorg, and asserts the rewind. In both cases, after the rejection the harness re-asserts the Dafny invariants: **rejecting a violation must leave the verifier model-valid.**

Composed fault injection.
On top of the invalid path, `armFault` derives one *transient*, one-shot fault from the seed and installs it on both the live `verifiedDB` (via a wrapping `faultVerifiedDB`) and every chain's engine wrapper (via `EngineWrappers().SetGate`). The fault targets a single method drawn from the WAL-backed `verifiedDB` calls (`Commit` , `Get` , `Rewind` , and the pending-transition accessors) or the engine apply path (`ForkchoiceUpdate` returning `context.DeadlineExceeded` , matching RewindEngine root cause 1; `NewPayload` rejecting a synthetic insert). It fires at most once, during a single production call, and the verifier must recover transparently via its normal `recordStep` / `observeStep` replay (WAL replay for DB targets, apply replay for engine targets), with no change to the eventual outcome. About one seed in seven arms no fault, so the plain fault-free path still runs. `fault.assertFiredIfReachable` guards against a fault that was reachable yet silently never fired (which would make the test vacuous).

Determinism and reproduction

Every layer is seeded solely from the fuzz-input bytes, so a corpus entry replays its exact chain set, violation choice, EL pre-state, and fault timing.
Failures reported by any target are reproducible by re-running the target against the saved input — no external services, no wall-clock or randomness outside the seeded PRNG.

Fuzzing File Coverage

Not addressed by client

The following is a table in brief listing the coverage percentages of each file in the covered modules. A more comprehensive listing broken down per function can be found in Fuzzing Coverage Index

FILE	COVERAGE
activity/interop/algo.go	54.2%
activity/interop/checker.go	75.0%
activity/interop/cycle.go	24.4%
activity/interop/interop.go	59.9%
activity/interop/log_backfill.go	0.0%
activity/interop/logdb.go	70.6%
activity/interop/reader.go	0.0%
activity/interop/types.go	100.0%
activity/interop/verification_view.go	75.0%
activity/interop/verified_db.go	78.4%
chain_container/chain_container.go	32.2%
chain_container/engine_controller/engine_controller.go	30.8%
chain_container/engine_controller/rewind.go	69.6%
chain_container/invalidation.go	38.0%
chain_container/super_authority.go	3.1%
chain_container/virtual_node/virtual_node.go	0.0%

Fuzzing Coverage Index

Not addressed by client

The following is a comprehensive listing of the statement coverage percentages of each function in the covered modules.

activity/interop/algo.go

Function	Coverage %
l1Inclusion	80.0%
verifyExecutingMessage	65.2%
verifyInteropMessages	44.0%

activity/interop/checker.go

Function	Coverage %
SameL1Chain	77.8%
newL1ConsistencyChecker	66.7%

activity/interop/cycle.go

Function	Coverage %
buildCycleGraph	40.0%
verifyCycleMessages	37.0%
checkCycle	12.5%
removeFromSlice	0.0%
executingMessageBefore	0.0%
collectCycleParticipants	0.0%
addNode	0.0%
addEdge	0.0%

activity/interop/interop.go

Function	Coverage %
init	100.0%
commitVerifiedResult	100.0%
checkChainsReady	100.0%
progressAndRecord	91.3%
collectCurrentL1	90.9%
progressInterop	88.9%
observeRound	88.1%
checkPreconditions	84.6%
firstVerifiableTimestamp	83.3%
String	83.3%
invalidateBlock	80.0%
decideVerifiedResult	80.0%
applyPendingTransition	77.4%
refreshCurrentL1OnWait	75.0%
newInvalidHead	75.0%
buildPendingTransition	75.0%
applyRewindPlan	74.4%
shouldResetEnginesOnRewind	71.4%
UnmarshalJSON	70.0%
Stop	70.0%
MarshalJSON	66.7%
New	65.2%
buildRewindPlan	65.0%
verify	53.3%
resetChainEnginesIfNeeded	28.6%
waitForColdStartInit	0.0%
tryInitFromVerifiedDB	0.0%
runLoop	0.0%
progress	0.0%
VerifiedResultAtTimestamp	0.0%
VerifiedBlockAtL1	0.0%
VerifiedAtTimestamp	0.0%
Start	0.0%
Reset	0.0%
Name	0.0%
LatestVerifiedL2Block	0.0%
IsActiveAt	0.0%
CurrentL1	0.0%

activity/interop/log_backfill.go

Function	Coverage %
advanceColdStartInit	0.0%
collectFirstSafeHeadTimestamps	0.0%
runColdStartBackfill	0.0%
backfillChain	0.0%
reconcileLogsDBTail	0.0%

activity/interop/logdb.go

Function	Coverage %
processBlockLogs	77.3%
persistFrontierLogs	75.0%
openLogsDB	75.0%
verifyCanAddTimestamp	71.4%
sealFetchedBlockIntoLogsDB	71.4%
sealBlockDataIntoLogsDB	60.9%

activity/interop/reader.go

Function	Coverage %
VerifiedResultAtTimestamp	0.0%

activity/interop/types.go

Function	Coverage %
IsValid	100.0%
IsEmpty	100.0%
ToVerifiedResult	100.0%

activity/interop/verification_view.go

Function	Coverage %
buildFrontierBlockView	100.0%
resolveFrontierVerificationView	80.0%
block	75.0%
contains	0.0%

activity/interop/verified_db.go

Function	Coverage %
timestampToKey	100.0%
concrete	100.0%
LastTimestamp	100.0%
FirstTimestamp	100.0%
Close	100.0%
ClearPendingTransition	100.0%
GetPendingTransition	94.4%
Rewind	92.9%
SetPendingTransition	87.5%
Commit	82.4%
Get	75.0%
initTimestampBounds	73.3%
OpenVerifiedDB	64.7%
Has	0.0%

chain_container/chain_container.go

Function	Coverage %
isCriticalRewindError	100.0%
getVN	100.0%
Resume	100.0%
Pause	100.0%
ID	100.0%
OptimisticAt	84.6%
LocalSafeBlockAtTimestamp	78.6%
RewindEngine	72.2%
safeDBAtL2	68.8%
PauseAndStopVN	66.7%
FetchReceipts	66.7%
BlockTime	66.7%
BlockNumberToTimestamp	60.0%
SyncStatus	55.6%
subPath	0.0%
setVN	0.0%
defaultVirtualNodeFactory	0.0%
attachInProcRollupClient	0.0%
VerifierCurrentL1s	0.0%
TimestampToBlockNumber	0.0%
Stop	0.0%
Start	0.0%
SetResetCallback	0.0%
RegisterVerifier	0.0%
OutputRootAtL2BlockHash	0.0%
OptimisticOutputAtTimestamp	0.0%
NewChainContainer	0.0%
FirstSafeHeadTimestamp	0.0%
ELFinalizedHead	0.0%

chain_container/engine_controller/engine_controller.go

Function	Coverage %
NewEngineControllerWithL2AndRollup	100.0%
L2BlockRefByNumber	100.0%
blockAtTimestamp	75.0%
blockNumberAtTimestamp	66.7%
FetchReceipts	66.7%
OutputV0AtBlockNumber	61.1%
OutputV0ByBlockHash	0.0%
NewEngineControllerWithL2	0.0%
NewEngineControllerFromConfig	0.0%
L2BlockRefByLabel	0.0%
Close	0.0%
BlockAtTimestamp	0.0%

chain_container/engine_controller/rewind.go

Function	Coverage %
earliest	100.0%
RewindToTimestamp	84.2%
insertSyntheticPayload	75.0%
computeRewindTargets	66.7%
verifyRewindState	62.5%
forkchoiceUpdate	60.0%
forkchoiceUpdateWithRetry	45.5%

chain_container/invalidation.go

Function	Coverage %
heightToKey	100.0%
encodeDenyRecords	100.0%
Close	100.0%
Add	88.2%
decodeDenyRecords	83.3%
InvalidateBlock	71.4%
OpenDenyList	69.2%
PruneDeniedAtOrAfterTimestamp	66.7%
HasDeniedAtOrAfterTimestamp	66.7%
HasDeniedAtOrAfterTimestamp	56.2%
PruneAtOrAfterTimestamp	33.3%
LastDeniedOutputV0	0.0%
GetOutputV0	0.0%
GetDeniedRecords	0.0%
GetDeniedHashes	0.0%
Contains	0.0%

chain_container/super_authority.go

Function	Coverage %
OutputV0AtBlockNumber	66.7%
localSafeTimestamp	0.0%
allVerifiersPreActivationAt	0.0%
IsDenied	0.0%
GetDeniedOutput	0.0%
FullyVerifiedL2Head	0.0%
FinalizedL2Head	0.0%

chain_container/virtual_node/virtual_node.go

Function	Coverage %
defaultInnerNodeFactory	0.0%
generateVirtualNodeID	0.0%
NewVirtualNode	0.0%
Start	0.0%
Stop	0.0%
State	0.0%
SafeHeadAtL1	0.0%
FirstSafeHeadEntry	0.0%
L1AtSafeHead	0.0%
SyncStatus	0.0%

Appendix: Full AI Analysis Report

Not addressed by client

Dafny Spec Check Report

> **Project**: `op-supernode`
> **Dafny model**: `dafny-models`
> **Generated**: 2026-07-01

Summary

#	Go Function	Dafny Method	Pre ✓	Pre ?	Pre ✗	Post ✓	Post ?	Post ✗
1	`persistFrontierLogs`	`Interop.PersistFrontierLogs`	5	0	0	1	4	0
2	`processBlockLogs`	`Interop.ProcessBlock`	1	5	0	0	1	1
3	`refreshCurrentL1OnWait`	`Interop.RefreshCurrentL1OnWait`	1	0	0	1	0	0
4	`verifyExecutingMessage`	`Interop.VerifyExecutingMessage`	2	0	0	2	1	0
5	`l1Inclusion`	`Interop.ComputeL1Inclusion`	1	0	0	1	0	0
6	`resetChainEnginesIfNeeded`	`Interop.RewindChainEngines`	7	0	0	6	0	0
7	`resolveFrontierVerificationView`	`Interop.ResolveFrontierVerificationView`	3	0	0	2	1	0
8	`checkChainsReady`	`Interop.CheckChainsReady`	0	3	0	4	1	0
9	`buildPendingTransition`	`Interop.BuildPendingTransition`	6	0	0	3	0	3
10	`verifyInteropMessages`	`Interop.VerifyMessages`	0	4	0	2	1	1
11	`applyRewindPlan`	`Interop.ApplyRewindPlan`	10	0	0	8	2	0
12	`verify`	`Interop.Verify`	2	2	0	4	0	0
13	`observeRound`	`Interop.ObserveRound`	2	0	0	3	0	3
14	`applyPendingTransition`	`Interop.ApplyPendingTransition`	6	0	0	1	8	0
15	`progressInterop`	`Interop.ProgressInterop`	3	0	0	6	2	0
16	`progressAndRecord`	`Interop.ProgressAndRecord`	0	4	0	4	0	0

1. `persistFrontierLogs` → `Interop.PersistFrontierLogs`

Go file: `supernode/activity/interop/logdb.go:82`
Dafny: `dafny-models/Interop.dfy` — class `Interop`, method `PersistFrontierLogs`
Notes: Go validates timestamp monotonicity and enforces a maximum gap size (no more than one block time between consecutive timestamps) inside sealBlockDataIntoLogsDB; the Dafny model uses assume for these parent-hash and monotonicity conditions. Go also iterates over a map (nondeterministic order), while the Dafny model sequences log persistence deterministically.

Preconditions

- ☑ `Valid()` — **Satisfied**
- ☑ `blocksAtTS.Keys == chains.Keys` — **Satisfied**
- ☑ `AdvancesAllLogsDBs(ts, blocksAtTS)` — **Satisfied**
- ☑ `BlocksExistedOnChain(blocksAtTS)` — **Satisfied**
- ☑ `AllLogsDBsConsistentWithChainData()` — **Satisfied**

Postconditions

- [?] `Valid()` — **Uncertain** — Go's processBlockLogs can return an I/O error with no analog in the Dafny model — ProcessBlock is axiomatically assumed to always succeed (ensures {:axiom} Valid()). If processBlockLogs for chain C fails after partially writing

to logsDBs[C] (e.g., block data written but seal not committed), the sub-predicates AllLogsDBsConsistentWithChainData() and VerifiedHeadsAreHighestBlocksUpToTimestamp() that compose Valid() may no longer hold for chain C. The ProcessBlock axiom only covers the success path and therefore does not bound this Go-specific failure mode.

- [?] `AdvancesAllLogsDBs(ts, blocksAtTS)` — **Uncertain** — If processBlockLogs fails in Go for chain C after partially updating logsDBs[C], logsDBs[C].LatestSealedBlock() may be left in an intermediate state — neither the old value (which satisfied AdvancesLogsDB by precondition 3) nor Some(blocksAtTS[C]) (which would satisfy it trivially). In that case AdvancesLogsDB(ts, C, blocksAtTS[C]) can fail the check latestBlock.number <= newBlock.number <= latestBlock.number + 1 evaluated in the post-state, violating AdvancesAllLogsDBs.
- [?] `AllLogsDBsConsistentWithChainData()` — **Uncertain** — Same root cause as post-conditions 1 and 2: if processBlockLogs fails in Go after writing a partial seal entry for chain C, FindSealedBlock may return a record whose on-chain block data (BlockInfo / BlockLogs) does not match what was committed to the logsDB, directly violating LogsDBConsistentWithChainData(C) and therefore AllLogsDBsConsistentWithChainData(). This failure path has no representation in the Dafny spec.
- ☑ `success ==> UpdatedAllLogsDBs(blocksAtTS)` — **Satisfied**
- [?] `!success ==> forall chainID :: chainID in logsDBs.Keys ==> UpdatedLogsDB(chainID, blocksAtTS[chainID]) || unchan` — **Uncertain** — If processBlockLogs for chain C returns a Go I/O error after partially modifying logsDBs[C] (a failure mode absent from the Dafny spec), neither disjunct holds: UpdatedLogsDB(C, blocksAtTS[C]) requires LatestSealedBlock() == Some(blocksAtTS[C]) and a specific framing on all other block numbers, which a half-written logsDB does not satisfy; unchanged(logsDBs[C]) requires every FindSealedBlock entry to equal its old value, which the partial write violates. Go's nondeterministic map iteration order also means the identity of chain C is non-deterministic, so this scenario can affect any element of logsDBs.Keys.

Callsite Analysis

Caller	Precondition Verdicts
`applyPendingTransition` (supernode/activity/interop/interop.go:757)	`Valid()` ✓ — applyPendingTransition is the Go body of ApplyPendingTransition, whose Dafny spec carries Valid() as a precondition. The DecisionAdvance branch performs no state-mutating operations before line 757 (only a nil-guard on pending.Result); Valid() is therefore intact at the call site.; `blocksAtTS.Keys == chains.Keys` ✓ — ValidPendingTransition(pending) — implied by Valid() (a precondition of ApplyPendingTransition) — requires pending.result.value.l2Heads.Keys == CHAIN_IDS for an Advance decision. Valid() also requires chains.Keys == CHAIN_IDS. Together they give pending.Result.L2Heads.Keys == i.chains.Keys, which is exactly blocksAtTS.Keys == chains.Keys at the call site.; `AdvancesAllLogsDBs(ts, blocksAtTS)` ✓ — PendingTransitionIsConsistent() is a precondition of ApplyPendingTransition. For a stored Advance pending transition, TransitionConsistentWithLogs(pending) holds, which directly asserts AdvancesAllLogsDBs(pending.result.value.timestamp, pending.result.value.l2Heads). These are the exact ts and blocksAtTS values passed to persistFrontierLogs. No logsDB mutations intervene between ApplyPendingTransition entry and line 757.; `BlocksExistedOnChain(blocksAtTS)` ✓ — PendingTransitionIsConsistent() is a precondition of ApplyPendingTransition. For a stored Advance transition, TransitionConsistentWithChainState(pending) holds, which asserts BlocksExistedOnChain(pending.result.value.l2Heads). That map is identical to blocksAtTS at the call site. No chain-state mutations occur before line 757.; `AllLogsDBsConsistentWithChainData()` ✓ — AllLogsDBsConsistentWithChainData() is a direct precondition of ApplyPendingTransition. The DecisionAdvance branch writes to no logsDB before reaching line 757 (only a nil-guard and the pending.Result nil-check execute), so the invariant is preserved intact at the call site.

Violation Scenarios

Postcondition `Valid()` (uncertain):

Go's processBlockLogs can return an I/O error with no analog in the Dafny model — ProcessBlock is axiomatically assumed to always succeed (ensures {:axiom} Valid()). If processBlockLogs for chain C fails after partially writing to logsDBs[C] (e.g., block data written but seal not committed), the sub-predicates AllLogsDBsConsistentWithChainData() and VerifiedHeadsAreHighestBlocksUpToTimestamp() that compose Valid() may no longer hold for chain C. The ProcessBlock axiom only covers the success path and therefore does not bound this Go-specific failure mode.

Postcondition `AdvancesAllLogsDBs(ts, blocksAtTS)` (uncertain):

If processBlockLogs fails in Go for chain C after partially updating logsDBs[C], logsDBs[C].LatestSealedBlock() may be left in an intermediate state — neither the old value (which satisfied AdvancesLogsDB by precondition 3) nor Some(blocksAtTS[C]) (which would satisfy it trivially). In that case AdvancesLogsDB(ts, C, blocksAtTS[C]) can fail the check latestBlock.number <= newBlock.number <= latestBlock.number + 1 evaluated in the post-state, violating AdvancesAllLogsDBs.

Postcondition `AllLogsDBsConsistentWithChainData()` (uncertain):

Same root cause as post-conditions 1 and 2: if processBlockLogs fails in Go after writing a partial seal entry for chain C, FindSealedBlock may return a record whose on-chain block data (BlockInfo / BlockLogs) does not match what was committed to the logsDB, directly violating LogsDBConsistentWithChainData(C) and therefore AllLogsDBsConsistentWithChainData(). This failure path has no representation in the Dafny spec.

Postcondition

`!success ==> forall chainID :: chainID in logsDBs.Keys ==> UpdatedLogsDB(chainID, blocksAtTS[chainID]) || unchanged(`
(uncertain):

If processBlockLogs for chain C returns a Go I/O error after partially modifying logsDBs[C] (a failure mode absent from the Dafny spec), neither disjunct holds: UpdatedLogsDB(C, blocksAtTS[C]) requires LatestSealedBlock() == Some(blocksAtTS[C]) and a specific framing on all other block numbers, which a half-written logsDB does not satisfy; unchanged(logsDBs[C]) requires every FindSealedBlock entry to equal its old value, which the partial write violates. Go's nondeterministic map iteration order also means the identity of chain C is non-deterministic, so this scenario can affect any element of logsDBs.Keys.

---

2. `processBlockLogs` → `Interop.ProcessBlock`

Go file: `supernode/activity/interop/logdb.go:222`
Dafny: `dafny-models/Interop.dfy` — class `Interop`, method `ProcessBlock`
Notes: The Dafny method is an axiom stub (ensures {:axiom} Valid(), ensures {:axiom} UpdatedLogsDB). In Go, `processBlockLogs` handles a special first-block case by sealing a virtual parent block when the logsDB is empty; it then iterates over all receipt logs, decodes executing messages when present, calls AddLog for each, and seals the block at the end. The parent-hash and block-number preconditions from the Dafny spec are enforced by the outer caller (`sealBlockDataIntoLogsDB`) rather than by `processBlockLogs` itself.

Preconditions

- ☑ `chainID in CHAIN_IDS` — **Satisfied**
- [?] `Valid()` — **Uncertain** — If sealBlockDataIntoLogsDB is invoked from a context where Valid() does not hold (e.g., mid-mutation by a concurrent or incompletely applied operation), Valid() would be absent at the processBlockLogs call site.
- [?] `BlockExistedOnChain(chainID, blockID)` — **Uncertain** — If sealBlockDataIntoLogsDB is called with a blockID that does not correspond to any block on chainID in the chain container (e.g., a speculative or incorrect block), BlockExistedOnChain(chainID, blockID) fails.
- [?] `info == chains[chainID].BlockInfo(blockID).value` — **Uncertain** — If the blockInfo supplied to sealBlockDataIntoLogsDB is stale, belongs to a different fork, or is otherwise non-canonical for blockID on chainID, the precondition fails.
- [?] `logs == chains[chainID].BlockLogs(blockID).value` — **Uncertain** — If the receipts parameter does not match the canonical BlockLogs for blockID on chainID (e.g., receipts from a reorged fork), the precondition fails.
- [?] `logsDBs[chainID].LatestSealedBlock().Some? ==> logsDBs[chainID].LatestSealedBlock().value.number + 1 == blockID.` — **Uncertain** — Under non-canonical block data (preconditions 3/4 not guaranteed), blockID.Number could exceed latestBlock.Number by more than 1 while blockInfo.ParentHash() still equals latestBlock.Hash if blockInfo is malformed. The Go code lacks an explicit numeric check that blockID.Number == latestBlock.Number + 1.

Postconditions

- [?] `{:axiom} Valid()` — **Uncertain** — processBlockLogs calls db.AddLog for each receipt log and db.SealBlock twice in the first-block case (virtual parent plus blockID). These operations modify one chain's logsDB. Whether all Valid() invariants are preserved — in particular AllLogsDBsConsistentWithChainData (requiring block seals to reflect canonical chain data),

VerifiedHeadsAreHighestBlocksUpToTimestamp, and logsDBs distinctness — depends on the correctness of the LogsDB implementation and the canonicality of the supplied blockInfo and receipts. This cannot be verified from the shown code.

- ☐ `{:axiom} UpdatedLogsDB(chainID, blockID)` — **Violated** — UpdatedLogsDB requires `forall n :: n != blockID.number ==> db.FindSealedBlock(n) == old(db.FindSealedBlock(n))`. In the first-block case (isFirstBlock == true && blockNum > 0), processBlockLogs calls db.SealBlock(common.Hash{}, parentBlock, blockInfo.Time()) at approximately line 242 before iterating logs. This seals the virtual parent at blockNum-1. Because the logsDB is empty before the call, old(db.FindSealedBlock(blockNum-1)) == None, but after the call FindSealedBlock(blockNum-1) == Some(parentBlock seal). Since blockNum-1 != blockID.number (= blockNum), this change violates the forall constraint, falsifying UpdatedLogsDB.

Callsite Analysis

Caller	Precondition Verdicts
`sealBlockDataIntoLogsDB` (supernode/activity/interop/logdb.go:143)	`chainID in CHAIN_IDS` ✓ — sealBlockDataIntoLogsDB guards with `chain, ok := i.chains[chainID]; if !ok { return nil }` before reaching the process called when chainID ∈ i.chains. Valid() guarantees chains.Keys == CHAIN_IDS, so chainID ∈ CHAIN_IDS holds at that point.; `Valid()` ? — sealBlockDataIntol calling processBlockLogs. The code between function entry and the call modifies no Interop state, so Valid() is preserved if it held at entry. However, the callers o Valid() holds at entry cannot be determined from the shown code.; `BlockExistedOnChain(chainID, blockID)` ? — sealBlockDataIntoLogsDB receives blockInfo chains[chainID].BlockInfo(blockID) is made to verify the block exists on the canonical chain. Satisfaction depends entirely on callers of sealBlockDataIntoLogsDB `info == chains[chainID].BlockInfo(blockID).value` ? — sealBlockDataIntoLogsDB passes the blockInfo parameter directly to processBlockLogs without ve mapping between the Go blockInfo argument and the canonical chain block info is not checked inside the shown caller.; `logs == chains[chainID].BlockLogs(b` the receipts parameter directly to processBlockLogs without comparing it to i.chains[chainID].BlockLogs(blockID). Correctness depends on callers of sealBlockDa data.; `logsDBs[chainID].LatestSealedBlock().Some? ==> logsDBs[chainID].LatestSealedBlock().value.number + 1 == blockID.number && info.parentHash =` ? — When hasBlocks is true, sealBlockDataIntoLogsDB explicitly checks `blockInfo.ParentHash() != latestBlock.Hash` and returns an error if mismatched, e number+1 relationship is not checked directly: the code only establishes latestBlock.Number < blockID.Number (by eliminating the > and == cases). The exact +1 (blockInfo.ParentHash() being the hash of block blockID.Number-1), which depends on uncertain preconditions 3 and 4 holding.

Violation Scenarios

Precondition `Valid()` (uncertain):

If sealBlockDataIntoLogsDB is invoked from a context where Valid() does not hold (e.g., mid-mutation by a concurrent or incompletely applied operation), Valid() would be absent at the processBlockLogs call site.

Precondition `BlockExistedOnChain(chainID, blockID)` (uncertain):

If sealBlockDataIntoLogsDB is called with a blockID that does not correspond to any block on chainID in the chain container (e.g., a speculative or incorrect block), BlockExistedOnChain(chainID, blockID) fails.

Precondition `info == chains[chainID].BlockInfo(blockID).value` (uncertain):

If the blockInfo supplied to sealBlockDataIntoLogsDB is stale, belongs to a different fork, or is otherwise non-canonical for blockID on chainID, the precondition fails.

Precondition `logs == chains[chainID].BlockLogs(blockID).value` (uncertain):

If the receipts parameter does not match the canonical BlockLogs for blockID on chainID (e.g., receipts from a reorged fork), the precondition fails.

Precondition
`logsDBs[chainID].LatestSealedBlock().Some? ==> logsDBs[chainID].LatestSealedBlock().value.number + 1 == blockID.numb`
(uncertain):

Under non-canonical block data (preconditions 3/4 not guaranteed), blockID.Number could exceed latestBlock.Number by more than 1 while blockInfo.ParentHash() still equals latestBlock.Hash if blockInfo is malformed. The Go code lacks an explicit numeric check that blockID.Number == latestBlock.Number + 1.

Postcondition `{:axiom} Valid()` (uncertain):

processBlockLogs calls db.AddLog for each receipt log and db.SealBlock twice in the first-block case (virtual parent plus blockID). These operations modify one chain's logsDB. Whether all Valid() invariants are preserved — in particular AllLogsDBsConsistentWithChainData (requiring block seals to reflect canonical chain data), VerifiedHeadsAreHighestBlocksUpToTimestamp, and logsDBs distinctness — depends on the correctness of the LogsDB implementation and the canonicality of the supplied blockInfo and receipts. This cannot be verified from the shown code.

Postcondition `{:axiom} UpdatedLogsDB(chainID, blockID)` (violated):

UpdatedLogsDB requires `forall n :: n != blockID.number ==> db.FindSealedBlock(n) == old(db.FindSealedBlock(n))` . In the first-block case (isFirstBlock == true && blockNum > 0), processBlockLogs calls db.SealBlock(common.Hash{}, parentBlock, blockInfo.Time()) at approximately line 242 before iterating logs. This seals the virtual parent at blockNum-1. Because the logsDB is empty before the call, old(db.FindSealedBlock(blockNum-1)) == None, but after the call FindSealedBlock(blockNum-1) == Some(parentBlock seal). Since blockNum-1 != blockID.number (= blockNum), this change violates the forall constraint, falsifying UpdatedLogsDB.

3. `refreshCurrentL1OnWait` → `Interop.RefreshCurrentL1OnWait`

Go file: `supernode/activity/interop/interop.go:496`
Dafny: `dafny-models/Interop.dfy` — class `Interop` , method `RefreshCurrentL1OnWait`
Notes: The Dafny method is an axiom stub (ensures {:axiom} Valid()). In Go, `refreshCurrentL1OnWait` calls `collectCurrentL1` and updates `currentL1` under a mutex lock. If collection fails, the error is logged and the function returns gracefully with (false, nil); this silent-failure path is absent from the Dafny model.

Preconditions

- ☑ `Valid()` — **Satisfied**

Postconditions

- ☑ `{:axiom} Valid()` — **Satisfied**

Callsite Analysis

Caller	Precondition Verdicts
`progressAndRecord` (supernode/activity/interop/interop.go:477)	`Valid()` ✓ — progressAndRecord has Valid() as its own precondition. The only call that modifies state before reaching line 477 is progressInterop(), whose postcondition ensures Valid() (modifies only chains.Values). The metrics increment between progressInterop's return and the refreshCurrentL1OnWait call is not in Valid()'s reads clause (reads verifiedDB, logsDBs.Values), so Valid() remains unaffected. The pending != nil early-return path bypasses refreshCurrentL1OnWait entirely. The err != nil early-return after progressInterop also bypasses it. Therefore, at line 477, Valid() is guaranteed by progressInterop's postcondition.

4. `verifyExecutingMessage` → `Interop.VerifyExecutingMessage`

Go file: `supernode/activity/interop/algo.go:184`
Dafny: `dafny-models/Interop.dfy` — class `Interop` , method `VerifyExecutingMessage`
Notes: This function is also a co_function of the VerifyMessages mapping entry; here it is checked directly against its own Dafny spec. In Go, the function returns error (nil = valid, non-nil = invalid) rather than bool. Go performs two separate ErrUnknownChain checks (for the source logsDB and for both chains map entries) before the activation checks; the Dafny model uses a single combined guard. Same-timestamp messages are checked against the frontier view first in both models.

Preconditions

- ☑ `Valid()` — **Satisfied**
- ☑ `executingChain in CHAIN_IDS` — **Satisfied**

Postconditions

- ☑ `Valid()` — **Satisfied**
- ☑ `valid ==> ValidExecutingMessage(executingTimestamp, executingChain, execMsg)` — **Satisfied**
- [?]
  `valid ==> if execMsg.timestamp == executingTimestamp then InitMsgInFrontierView(execMsg, view) else assert execM`

— **Uncertain** — execMsg.Timestamp == executingTimestamp (line 233), view.contains(execMsg.ChainID, query) returns ok=false (e.g., the frontier block for execMsg.ChainID has a different block number), but i.logsDBs[execMsg.ChainID] contains a sealed block at execMsg.BlockNum with timestamp executingTimestamp and matching checksum. sourceDB.Contains(query) at line 240 returns nil, so the function returns nil (valid), yet InitMsgInFrontierView(execMsg, view) is false. Under Valid() alone, VerifiedHeadsAreHighestBlocksUpToTimestamp() only requires logsDB blocks above the verified head to have timestamps strictly greater than the verified timestamp, which permits a block with timestamp == executingTimestamp. The scenario is excluded in practice by the caller's AdvancesAllLogsDBs(ts, blocksAtTS) + AllDBsInSync() invariants (ensuring all logsDB blocks carry timestamps < executingTimestamp), but those are preconditions of VerifyMessages, not of VerifyExecutingMessage itself.

Callsite Analysis

Caller	Precondition Verdicts
`verifyInteropMessages` (supernode/activity/interop/algo.go:151)	`Valid()` ✓ — VerifyMessages requires Valid() and has no modifies clause (frame-safe), so Valid() holds throughout the caller's execution up to and including the call at line 151. No mutations of Interop state occur before this call site.; `executingChain in CHAIN_IDS` ✓ — The argument is chainID, drawn from iterating blocksAtTimestamp. VerifyMessages requires blocksAtTS.Keys == CHAIN_IDS, so every chainID in the iteration is guaranteed to be in CHAIN_IDS.

Violation Scenarios

Postcondition

`valid ==> if execMsg.timestamp == executingTimestamp then InitMsgInFrontierView(execMsg, view) else assert execMsg.t` (uncertain):

execMsg.Timestamp == executingTimestamp (line 233), view.contains(execMsg.ChainID, query) returns ok=false (e.g., the frontier block for execMsg.ChainID has a different block number), but i.logsDBs[execMsg.ChainID] contains a sealed block at execMsg.BlockNum with timestamp executingTimestamp and matching checksum. sourceDB.Contains(query) at line 240 returns nil, so the function returns nil (valid), yet InitMsgInFrontierView(execMsg, view) is false. Under Valid() alone, VerifiedHeadsAreHighestBlocksUpToTimestamp() only requires logsDB blocks above the verified head to have timestamps strictly greater than the verified timestamp, which permits a block with timestamp == executingTimestamp. The scenario is excluded in practice by the caller's AdvancesAllLogsDBs(ts, blocksAtTS) + AllDBsInSync() invariants (ensuring all logsDB blocks carry timestamps < executingTimestamp), but those are preconditions of VerifyMessages, not of VerifyExecutingMessage itself.

---

5. `l1Inclusion` → `Interop.ComputeL1Inclusion`

Go file: `supernode/activity/interop/algo.go:43`
Dafny: `dafny-models/Interop.dfy` — class `Interop`, method `ComputeL1Inclusion`
Notes: The Dafny method is an axiom stub (ensures {:axiom} Valid()). In Go, `l1Inclusion` iterates over `blocksAtTimestamp`, skips chains absent from `i.chains`, and selects the L1 block with the highest block number from `l1Heads`. If a chain is present in `blocksAtTimestamp` but absent from `l1Heads`, Go returns an error; the Dafny axiom stub does not model this failure path.

Preconditions

- ☑ `Valid()` — **Satisfied**

Postconditions

- ☑ `{:axiom} Valid()` — **Satisfied**

Callsite Analysis

Caller	Precondition Verdicts
`verifyInteropMessages` (supernode/activity/interop/algo.go:76)	`Valid()` ✓ — verifyInteropMessages maps to Interop.VerifyMessages, which has Valid() as its own precondition. The call to i.l1Inclusion is the first substantive operation in the body (line 76), executed before any state-mutating calls. VerifyMessages has no modifies clause, so no receiver state can have changed before this call site.

6. `resetChainEnginesIfNeeded` → `Interop.RewindChainEngines`

Go file: `supernode/activity/interop/interop.go:905`
Dafny: `dafny-models/Interop.dfy` — class `Interop`, method `RewindChainEngines`
Notes: This function is also a co_function of the ApplyRewindPlan mapping entry; here it is checked directly against its own Dafny spec. In Go, `resetChainEnginesIfNeeded` returns early if `plan.ResetAllChainsTo == nil`, which corresponds to the `plan.resetAllChainsTo.None?` guard inside the Dafny loop body. Errors are accumulated via a `recordErr` callback in Go rather than a boolean `failedAny` flag as in Dafny.

Preconditions

- ☑ `Valid()` — **Satisfied**
- ☑ `ValidRewindPlan(plan)` — **Satisfied**
- ☑ `PlanConsistentWithVerified(plan)` — **Satisfied**
- ☑ `RewoundVerifiedDB(plan)` — **Satisfied**
- ☑ `AllVerifiedCrossValid()` — **Satisfied**
- ☑ `forall k :: k in chainIDs <==> k in chains.Keys` — **Satisfied**
- ☑ `forall k :: k in chainIDs ==> PlanConsistentWithLogs(plan, k)` — **Satisfied**

Postconditions

- ☑ `Valid()` — **Satisfied**
- ☑ `ValidRewindPlan(plan)` — **Satisfied**
- ☑ `RewoundVerifiedDB(plan)` — **Satisfied**
- ☑ `AllVerifiedCrossValid()` — **Satisfied**
- ☑ `forall k :: k in chainIDs ==> PlanConsistentWithLogs(plan, k)` — **Satisfied**
- ☑ `unchanged(logsDBs.Values)` — **Satisfied**

Callsite Analysis

Caller	Precondition Verdicts
`applyRewindPlan` (supernode/activity/interop/interop.go:876)	`Valid()` ✓ — applyRewindPlan's Dafny spec requires Valid() on entry. Before line 876, three operations occur: (1) verifiedDB.Rewind — VerifiedDB.Rewind spec maintains VerifiedDB.Valid() and preserves pendingTransition; under ValidRewindPlan the None case means rewindAtOrAfter <= ACTIVATION_TIMESTAMP, so all entries (each ts >= activationTimestamp >= rewindAtOrAfter) are removed, leaving an empty DB. (2) PruneDeniedAtOrAfterTimestamp — modifies chain containers; its spec has no Interop-level postcondition. (3) db.Clear() on all logsDBs — LogsDB.Clear ensures LatestSealedBlock()==None, making AllLogsDBsConsistentWithChainData() and VerifiedHeadsAreHighestBlocksUpToTimestamp() vacuously true over the now-empty verifiedDB. Valid() is maintained.; `ValidRewindPlan(plan)` ✓ — applyRewindPlan's Dafny spec lists ValidRewindPlan(plan) as a precondition. The plan is a Go value type passed through unchanged; no intermediate operation modifies it or the constants (ACTIVATION_TIMESTAMP, CHAIN_IDS) on which ValidRewindPlan depends.; `PlanConsistentWithVerified(plan)` ✓ — applyRewindPlan requires PlanConsistentWithVerified(plan). For the None case (the only valid case when TargetHeads is nil under ValidRewindPlan), the predicate's antecedent plan.resetAllChainsTo.Some? is false, so the predicate holds trivially after the verifiedDB.Rewind. No logsDB modifications affect verifiedDB state.; `RewoundVerifiedDB(plan)` ✓ — applyRewindPlan's preconditions establish verifiedDB.pendingTransition.Some? with decision==Rewind and rewind==Some(plan). VerifiedDB.Rewind spec guarantees pendingTransition == old(pendingTransition), so the decision/rewind fields are preserved. For the None case, rewindAtOrAfter <= ACTIVATION_TIMESTAMP, so VerifiedDB.Rewind removes all entries (all ts >= activationTimestamp >= rewindAtOrAfter), giving
`applyRewindPlan` (supernode/activity/interop/interop.go:900)	`Valid()` ✓ — applyRewindPlan's Dafny spec requires Valid() on entry. Before line 900, verifiedDB.Rewind retains entries k < rewindAtOrAfter and preserves VerifiedDB.Valid(). PruneDeniedAtOrAfterTimestamp has no Interop.Valid() side-effects per its spec. db.Rewind(expectedHead) for selective chains — LogsDB.Rewind spec preserves FindSealedBlock(n) for all n <= expectedHead.number = plan.targetHeads[chainID].number, so AllLogsDBsConsistentWithChainData() is maintained for retained blocks; removed blocks can no longer trigger the forall. VerifiedHeadsAreHighestBlocksUpToTimestamp() is preserved because retained logsDB blocks had timestamps satisfying the predicate before the rewind (part of Valid() pre-call) and no new blocks are added.; `ValidRewindPlan(plan)` ✓ — Same reasoning as line 876; sortedChainIDs is a derived slice and does not alter plan or the constants ValidRewindPlan reads.; `PlanConsistentWithVerified(plan)` ✓ — For the Some(ts) case with ts = rewindAtOrAfter-1, VerifiedDB.Rewind(rewindAtOrAfter) postcondition retains all k < rewindAtOrAfter, so ts is still in the DB. PlanConsistentWithVerified requires ts to be the max key below rewindAtOrAfter (established by the precondition), which holds after the rewind since no new entries are introduced. plan.targetHeads == verifiedDB.Get(ts).l2Heads is unchanged. The subsequent logsDB operations do not modify verifiedDB.; `RewoundVerifiedDB(plan)` ✓ — pendingTransition is preserved by VerifiedDB.Rewind. For the Some(ts) case with ts = rewindAtOrAfter-1, after verifiedDB.Rewind(rewindAtOrAfter) the DB contains exactly {k : k < rewindAtOrAfter}; its MaxKey is ts (because PlanConsistentWithVerified guaranteed ts was the max key below rewindAtOrAfter), so verifiedDB.LastTimestamp()==Some(ts). plan.targetHeads == verifiedDB.Get(ts).l2Heads is unchanged. logsDB operations do not touch verifiedDB.; `AllVerifiedCrossValid()` ✓ — AllVerifiedCrossValid() was established as a precondition of applyRewindPlan. After verifiedDB.Rewind, only entries with ts < rewindAtOrAfter remain; each was already cross-valid by the precondition. db.Rewind(expectedHead) preserves all blocks with number <= plan.targetHeads[chainID].number per LogsDB.Rewind spec. Since verified l2Heads at each remaining timestamp have block numbers <= plan.targetHeads[chainID].number (l2Heads are non-decreasing in the verifiedDB by Valid()), all InitMsgInLogsDB witnesses for those timestamps are preserved. Cross-validity therefore holds for all retained timestamps.; `forall k :: k in chainIDs <==> k in chains.Keys` ✓ — The same sortedChainIDs slice is reused; the construction guarantee is identical and no intermediate step modifies i.chains keys.; `forall k :: k in chainIDs ==> PlanConsistentWithLogs(plan, k)` ✓ — applyRewindPlan's precondition plan.resetAllChainsTo.Some? ==> PlanConsistentWithAllLogs(plan) establishes FindSealedBlock(plan.targetHeads[chainID].number).Some? for every chain at the start. For chains where db.Rewind(expectedHead) is called (expectedHead == plan.TargetHeads[chainID]), LogsDB.Rewind spec preserves FindSealedBlock(n) for all n <= expectedHead.number, so PlanConsistentWithLogs is maintained. For chains skipped because latestBlock==expectedHead, the logsDB is unchanged and the condition trivially holds. The case latestBlock.Number < expectedHead.Number cannot occur under valid preconditions: it would require FindSealedBlock(expectedHead.number)==None, contradicting PlanConsistentWithAllLogs.

7. `resolveFrontierVerificationView` → `Interop.ResolveFrontierVerificationView`

Go file: `supernode/activity/interop/verification_view.go:29`
Dafny: `dafny-models/Interop.dfy` — class `Interop`, method `ResolveFrontierVerificationView`
Notes: The Dafny method is an axiom stub (ensures {:axiom} IsCorrectFrontierView). In Go, `resolveFrontierVerificationView` fetches block receipts for each chain in `blocksAtTS` via `chain.FetchReceipts`, extracts executing messages using `buildFrontierBlockView`, and builds a lookup map keyed by `frontierQueryKey`. Chains absent from `i.chains` are silently skipped in Go, while the Dafny precondition requires `blocksAtTS.Keys == CHAIN_IDS`.

Preconditions

- ☑ `Valid()` — **Satisfied**
- ☑ `blocksAtTS.Keys == CHAIN_IDS` — **Satisfied**
- ☑ `BlocksExistedOnChain(blocksAtTS)` — **Satisfied**

Postconditions

- ☑ `{:axiom} Valid()` — **Satisfied**
- ☑ `{:axiom} fresh(view)` — **Satisfied**
- [?] `{:axiom} IsCorrectFrontierView(view, blocksAtTS)` — **Uncertain** — Two gaps prevent confirmation. (1) FetchReceipts failure: the Dafny FetchReceipts spec guarantees only result.Some? ==> BlockInfo(blockID).Some?, not the converse; FetchReceipts can return None even when BlocksExistedOnChain(blocksAtTS) holds. In that case the Go function returns (nil, error) and produces no view, while the Dafny spec has no error return and unconditionally ensures a correctly-populated FrontierView — the postcondition cannot be discharged for that execution path. (2) buildFrontierBlockView opacity: in the success path, buildFrontierBlockView(chainID, blockInfo, receipts) at line ~42 must populate view.blocks[chainID] so that the abstract view.BlockInfo(chainID) and view.BlockLogs(chainID) ghost functions equal chains[chainID].BlockInfo(blocksAtTS[chainID]).value and chains[chainID].BlockLogs(blocksAtTS[chainID]).value respectively; the implementation of buildFrontierBlockView is not in scope, so this mapping cannot be confirmed. Note: the silent-skip branch (if !ok { continue }) is unreachable when preconditions hold because Valid() guarantees chains.Keys == CHAIN_IDS == blocksAtTS.Keys, so that branch is not a source of violation.

Callsite Analysis

Caller	Precondition Verdicts
`verify` (supernode/activity/interop/interop.go:601)	`Valid()` ✓ — Interop.Verify lists Valid() as its first requires clause. verify is the Go implementation of Verify; it passes blocksAtTS unchanged to resolveFrontierVerificationView without any intervening mutations, so Valid() holds at the call site.; `blocksAtTS.Keys == CHAIN_IDS` ✓ — Interop.Verify requires blocksAtTS.Keys == CHAIN_IDS. verify at line 601 forwards the same blocksAtTS map directly to resolveFrontierVerificationView without modification, so the key-set equality holds at the call site.; `BlocksExistedOnChain(blocksAtTS)` ✓ — Interop.Verify requires BlocksExistedOnChain(blocksAtTS). verify passes the same blocksAtTS to resolveFrontierVerificationView at line 601 without any modification, so BlocksExistedOnChain holds for every chainID in the map.

Violation Scenarios

Postcondition `{:axiom} IsCorrectFrontierView(view, blocksAtTS)` (uncertain):

Two gaps prevent confirmation. (1) FetchReceipts failure: the Dafny FetchReceipts spec guarantees only result.Some? ==> BlockInfo(blockID).Some?, not the converse; FetchReceipts can return None even when BlocksExistedOnChain(blocksAtTS) holds. In that case the Go function returns (nil, error) and produces no view, while the Dafny spec has no error return and unconditionally ensures a correctly-populated FrontierView — the postcondition cannot be discharged for that execution path. (2) buildFrontierBlockView opacity: in the success path, buildFrontierBlockView(chainID, blockInfo, receipts) at line ~42 must populate view.blocks[chainID] so that the abstract view.BlockInfo(chainID) and view.BlockLogs(chainID) ghost functions equal chains[chainID].BlockInfo(blocksAtTS[chainID]).value and chains[chainID].BlockLogs(blocksAtTS[chainID]).value respectively; the implementation of buildFrontierBlockView is not in scope, so this mapping cannot be confirmed. Note: the silent-skip branch (if !ok {

continue }) is unreachable when preconditions hold because Valid() guarantees chains.Keys == CHAIN_IDS == blocksAtTS.Keys, so that branch is not a source of violation.

---

8. `checkChainsReady` → `Interop.CheckChainsReady`

Go file: `supernode/activity/interop/interop.go:941`
Dafny: `dafny-models/Interop.dfy` — class `Interop`, method `CheckChainsReady`
Notes: In Go, all chains are queried concurrently via goroutines, with results collected through a buffered channel before returning; the Dafny model uses a sequential for loop over chainIDs. Go returns an error (wrapping ethereum.NotFound) when any chain is not ready, while the Dafny model returns None for the optional result type.

Preconditions

- [?] `Valid()` — **Uncertain** — In the uninitialized branch of observeRound, firstVerifiableTimestamp() is called before checkChainsReady. Because firstVerifiableTimestamp is not in scope (no Dafny spec provided), any modification it makes to the Interop struct could break Valid() before line 558 is reached.
- [?] `AllDBsInSync()` — **Uncertain** — Same as Valid(): firstVerifiableTimestamp() is not covered by any in-scope Dafny spec, so its effect on AllDBsInSync() cannot be confirmed. A modification to verifiedDB or any logsDB inside that function would break AllDBsInSync() before line 558.
- [?] `forall k :: k in logsDBs && logsDBs[k].LatestSealedBlock().None? ==> ts == activationTimestamp` — **Uncertain** — When the verifiedDB is empty (initialized=false), all logsDBs have LatestSealedBlock()==None. obs.NextTimestamp is assigned from firstVerifiableTimestamp(), which is not mapped to any in-scope Dafny spec. If firstVerifiableTimestamp() returns a value other than activationTimestamp (e.g. due to some offset or configuration), the precondition forall k :: LatestSealedBlock(k)==None ==> ts==activationTimestamp is violated.

Postconditions

- ☑ `Valid()` — **Satisfied**
- ☑ `result.Some? ==> result.value.blocks.Keys == chains.Keys` — **Satisfied**
- [?] `result.Some? ==> AdvancesAllLogsDBs(ts, result.value.blocks)` — **Uncertain** — AdvancesLogsDB requires logsDBs[chainID].LatestSealedBlock().value.number <= newBlock.number <= logsDBs[chainID].LatestSealedBlock().value.number + 1 for the Some case. The Dafny spec for OptimisticAt only guarantees BlockInfo(l2Block).Some? and BlockInfo(l2Block).value.timestamp <= ts; it provides no bound on l2Block.number relative to the logsDB's latest sealed block number. If OptimisticAt returns a block whose number is more than one step ahead of or behind the logsDB tip, AdvancesLogsDB is violated and the postcondition fails.
- ☑ `result.Some? ==> BlocksExistedOnChain(result.value.blocks)` — **Satisfied**
- ☑ `result.Some? ==> FrontierBlocksConsistentWithTimestamp(ts, result.value.blocks)` — **Satisfied**

Callsite Analysis

Caller	Precondition Verdicts
`observeRound` (supernode/activity/interop/interop.go:558)	`Valid()` ? — ObserveRound's spec requires Valid() at entry. Before line 558, the function calls verifiedDB.LastTimestamp() and verifiedDB.Get(lastTS), both frame-safe (no modifies clause), so they cannot break Valid(). However, in the uninitialized branch (verifiedDB empty), firstVerifiableTimestamp() is called and is not an in-scope mapped function; its modifies footprint and effect on Interop state are unknown. If it mutates verifiedDB, logsDBs, or Interop fields, Valid() may no longer hold at line 558.; `AllDBsInSync()` ? — ObserveRound's spec requires AllDBsInSync() at entry. Frame-safe verifiedDB calls before line 558 cannot break AllDBsInSync(). But firstVerifiableTimestamp() in the uninitialized branch is not in scope; if it advances or alters the verifiedDB or logsDBs independently, AllDBsInSync() may be violated before the call to checkChainsReady at line 558.; `forall k :: k in logsDBs && logsDBs[k].LatestSealedBlock().None? ==> ts == activationTimestamp` ? — When initialized=true: verifiedDB.LastTimestamp().Some? and AllDBsInSync() together imply logsDBs[k].LatestSealedBlock() == Some(...) for all k, so the antecedent is universally false and the condition holds vacuously. When initialized=false: AllDBsInSync() with verifiedDB.LastTimestamp()==None implies all logsDBs have LatestSealedBlock()==None, making the antecedent true for every k. ts is then set to the return value of firstVerifiableTimestamp(), which must equal activationTimestamp for the condition to hold. firstVerifiableTimestamp() is not in scope; it is not confirmed to return activationTimestamp.

Violation Scenarios

Precondition `Valid()` (uncertain):

In the uninitialized branch of observeRound, firstVerifiableTimestamp() is called before checkChainsReady. Because firstVerifiableTimestamp is not in scope (no Dafny spec provided), any modification it makes to the Interop struct could break Valid() before line 558 is reached.

Precondition `AllDBsInSync()` (uncertain):

Same as Valid(): firstVerifiableTimestamp() is not covered by any in-scope Dafny spec, so its effect on AllDBsInSync() cannot be confirmed. A modification to verifiedDB or any logsDB inside that function would break AllDBsInSync() before line 558.

Precondition `forall k :: k in logsDBs && logsDBs[k].LatestSealedBlock().None? ==> ts == activationTimestamp` (uncertain):

When the verifiedDB is empty (initialized=false), all logsDBs have LatestSealedBlock()==None. obs.NextTimestamp is assigned from firstVerifiableTimestamp(), which is not mapped to any in-scope Dafny spec. If firstVerifiableTimestamp() returns a value other than activationTimestamp (e.g. due to some offset or configuration), the precondition forall k :: LatestSealedBlock(k)==None ==> ts==activationTimestamp is violated.

Postcondition `result.Some? ==> AdvancesAllLogsDBs(ts, result.value.blocks)` (uncertain):

AdvancesLogsDB requires logsDBs[chainID].LatestSealedBlock().value.number <= newBlock.number <= logsDBs[chainID].LatestSealedBlock().value.number + 1 for the Some case. The Dafny spec for OptimisticAt only guarantees BlockInfo(l2Block).Some? and BlockInfo(l2Block).value.timestamp <= ts; it provides no bound on l2Block.number relative to the logsDB's latest sealed block number. If OptimisticAt returns a block whose number is more than one step ahead of or behind the logsDB tip, AdvancesLogsDB is violated and the postcondition fails.

---

9. `buildPendingTransition` → `Interop.BuildPendingTransition`

Go file: `supernode/activity/interop/interop.go:651`
Dafny: `dafny-models/Interop.dfy` — class `Interop`, method `BuildPendingTransition`
Notes: Go uses firstVerifiableTimestamp (which may exceed activationTimestamp after a cold start with late-joining chains) as the lower bound in buildRewindPlan, while the Dafny model always uses activationTimestamp. Go also sets resetAllChainsTo based on a per-chain deny-list check (HasDeniedAtOrAfterTimestamp); the Dafny model treats this field as an opaque decision with no link to a deny-list predicate.

Preconditions

- ☑ `Valid()` — **Satisfied**
- ☑ `!output.WaitOutput?` — **Satisfied**
- ☑ `ValidStepOutput(output, obs)` — **Satisfied**
- ☑ `OutputConsistentWithVerified(output, obs)` — **Satisfied**
- ☑ `OutputConsistentWithLogs(output, obs)` — **Satisfied**
- ☑ `OutputConsistentWithChainState(output, obs)` — **Satisfied**

Postconditions

- ☐ `ValidPendingTransition(pendingTx)` — **Violated** — In the DecisionRewind branch, buildRewindPlan can produce plans that fail ValidRewindPlan, which is a conjunct of ValidPendingTransition. Two concrete paths: (1) shouldResetEnginesOnRewind returns false and *obs.LastVerifiedTS > activationTimestamp — plan.ResetAllChainsTo is nil (Dafny None), but ValidRewindPlan's None branch requires rewindAtOrAfter <= ACTIVATION_TIMESTAMP; since rewindAtOrAfter = lastTS > activationTimestamp, the condition fails. (2) shouldResetEnginesOnRewind returns true and lastTS <= firstVerifiableTimestamp — buildRewindPlan sets plan.ResetAllChainsTo = Some(lastTS−1) but returns before setting plan.TargetHeads (empty map); ValidRewindPlan's Some branch requires targetHeads.Keys == CHAIN_IDS, which an empty map cannot satisfy. The Advance and Invalidate branches are

unaffected: ValidStepOutput (precondition 3) directly supplies |invalidHeads|==0 for Advance, 0<|invalidHeads| for Invalidate, and l2Heads.Keys==CHAIN_IDS for both.

- ☐ `TransitionConsistentWithVerified(pendingTx)` — **Violated** — Advance maps to AdvancesVerifiedDB, which is guaranteed by precondition OutputConsistentWithVerified (AdvanceOutput case). Invalidate is trivially true. For Rewind, the obligation is PlanConsistentWithVerified, which fires only when resetAllChainsTo.Some?. When shouldResetEnginesOnRewind=true and lastTS <= firstVerifiableTimestamp: plan.ResetAllChainsTo = Some(lastTS−1) but plan.TargetHeads is empty; PlanConsistentWithVerified requires plan.targetHeads == verifiedDB.Get(lastTS−1).l2Heads, but verifiedDB.Get returns a map with Keys==CHAIN_IDS (from Valid()) while plan.TargetHeads is empty. Additionally, if lastTS == activationTimestamp, verifiedDB.Has(lastTS−1) = verifiedDB.Has(activationTimestamp−1) is false because Valid() guarantees all db entries >= activationTimestamp, again violating PlanConsistentWithVerified.
- ☐ `TransitionConsistentWithLogs(pendingTx)` — **Violated** — Advance requires AdvancesAllLogsDBs, which is established by precondition OutputConsistentWithLogs (AdvanceOutput case), and newL2Heads.Keys==logsDBs.Keys follows from ValidStepOutput and Valid(). Invalidate is trivially true. For Rewind, TransitionConsistentWithLogs requires: for every k in logsDBs.Keys, if resetAllChainsTo.Some? then k in pending.rewind.value.targetHeads and PlanConsistentWithLogs holds. When shouldResetEnginesOnRewind=true and lastTS <= firstVerifiableTimestamp, buildRewindPlan returns with plan.TargetHeads empty; the antecedent resetAllChainsTo.Some? is true (plan.ResetAllChainsTo = Some(lastTS−1)), so k in targetHeads is required for every chain k in CHAIN_IDS, but an empty map satisfies this for no chain — violated for all CHAIN_IDS elements. When shouldResetEnginesOnRewind=true and lastTS > firstVerifiableTimestamp, plan.TargetHeads = prevResult.L2Heads whose keys are CHAIN_IDS (from Valid()), and PlanConsistentWithLogs follows from OutputConsistentWithLogs (RewindOutput with activationTimestamp < lastTS).
- ☑ `output.AdvanceOutput? <==> pendingTx.decision.Advance?` — **Satisfied**
- ☑ `output.InvalidateOutput? <==> pendingTx.decision.Invalidate?` — **Satisfied**
- ☑ `(pendingTx.decision.Advance? || pendingTx.decision.Invalidate?) ==> pendingTx.result.value == output.result` — **Satisfied**

Callsite Analysis

Caller	Precondition Verdicts
`progressAndRecord` (supernode/activity/interop/interop.go:483)	`Valid()` ✓ — ProgressAndRecord's own Dafny spec requires Valid() as a precondition. ProgressInterop (called immediately before buildPendingTransition) ensures Valid() in its postcondition, so Valid() is re-established at the callsite.; `!output.WaitOutput?` ✓ — Before reaching the buildPendingTransition call, progressAndRecord returns early when output.Decision == DecisionWait (the Go guard 'if output.Decision == DecisionWait { return i.refreshCurrentL1OnWait() }'). DecisionWait is therefore excluded from any call to buildPendingTransition.; `ValidStepOutput(output, obs)` ✓ — ProgressInterop's Dafny spec ensures ValidStepOutput(output, obs). This postcondition is established by the call to progressInterop immediately before buildPendingTransition is invoked in progressAndRecord.; `OutputConsistentWithVerified(output, obs)` ✓ — ProgressInterop's Dafny spec ensures OutputConsistentWithVerified(output, obs). This postcondition is established by the call to progressInterop immediately before buildPendingTransition is invoked.; `OutputConsistentWithLogs(output, obs)` ✓ — ProgressInterop's Dafny spec ensures OutputConsistentWithLogs(output, obs). This postcondition is established by the call to progressInterop immediately before buildPendingTransition is invoked.; `OutputConsistentWithChainState(output, obs)` ✓ — ProgressInterop's Dafny spec ensures OutputConsistentWithChainState(output, obs). This postcondition is established by the call to progressInterop immediately before buildPendingTransition is invoked.

Violation Scenarios

Postcondition `ValidPendingTransition(pendingTx)` (violated):

In the DecisionRewind branch, buildRewindPlan can produce plans that fail ValidRewindPlan, which is a conjunct of ValidPendingTransition. Two concrete paths: (1) shouldResetEnginesOnRewind returns false and *obs.LastVerifiedTS > activationTimestamp — plan.ResetAllChainsTo is nil (Dafny None), but ValidRewindPlan's None branch requires rewindAtOrAfter <= ACTIVATION_TIMESTAMP; since rewindAtOrAfter = lastTS > activationTimestamp, the condition fails. (2) shouldResetEnginesOnRewind returns true and lastTS <= firstVerifiableTimestamp — buildRewindPlan sets plan.ResetAllChainsTo = Some(lastTS−1) but returns before setting plan.TargetHeads (empty map); ValidRewindPlan's Some branch requires targetHeads.Keys == CHAIN_IDS, which an empty map cannot satisfy. The Advance and Invalidate branches are unaffected: ValidStepOutput (precondition 3) directly supplies |invalidHeads|==0 for Advance, 0<|invalidHeads| for Invalidate, and l2Heads.Keys==CHAIN_IDS for both.

Postcondition `TransitionConsistentWithVerified(pendingTx)` (violated):

Advance maps to AdvancesVerifiedDB, which is guaranteed by precondition OutputConsistentWithVerified (AdvanceOutput case). Invalidate is trivially true. For Rewind, the obligation is PlanConsistentWithVerified, which fires only when resetAllChainsTo.Some?. When shouldResetEnginesOnRewind=true and lastTS <= firstVerifiableTimestamp: plan.ResetAllChainsTo = Some(lastTS−1) but plan.TargetHeads is empty; PlanConsistentWithVerified requires plan.targetHeads == verifiedDB.Get(lastTS−1).l2Heads, but verifiedDB.Get returns a map with Keys==CHAIN_IDS (from Valid()) while plan.TargetHeads is empty. Additionally, if lastTS == activationTimestamp, verifiedDB.Has(lastTS−1) = verifiedDB.Has(activationTimestamp−1) is false because Valid() guarantees all db entries >= activationTimestamp, again violating PlanConsistentWithVerified.

Postcondition `TransitionConsistentWithLogs(pendingTx)` (violated):

Advance requires AdvancesAllLogsDBs, which is established by precondition OutputConsistentWithLogs (AdvanceOutput case), and newL2Heads.Keys==logsDBs.Keys follows from ValidStepOutput and Valid(). Invalidate is trivially true. For Rewind, TransitionConsistentWithLogs requires: for every k in logsDBs.Keys, if resetAllChainsTo.Some? then k in pending.rewind.value.targetHeads and PlanConsistentWithLogs holds. When shouldResetEnginesOnRewind=true and lastTS <= firstVerifiableTimestamp, buildRewindPlan returns with plan.TargetHeads empty; the antecedent resetAllChainsTo.Some? is true (plan.ResetAllChainsTo = Some(lastTS−1)), so k in targetHeads is required for every chain k in CHAIN_IDS, but an empty map satisfies this for no chain — violated for all CHAIN_IDS elements. When shouldResetEnginesOnRewind=true and lastTS > firstVerifiableTimestamp, plan.TargetHeads = prevResult.L2Heads whose keys are CHAIN_IDS (from Valid()), and PlanConsistentWithLogs follows from OutputConsistentWithLogs (RewindOutput with activationTimestamp < lastTS).

---

10. `verifyInteropMessages` → `Interop.VerifyMessages`

Go file: `supernode/activity/interop/algo.go:69`

Dafny: `dafny-models/Interop.dfy` — class `Interop`, method `VerifyMessages`

Notes: In Go, executing messages in the first sealed block are silently skipped because no predecessor frontier entry exists for that block. A chain is also silently skipped if it is absent from i.logsDBs. Additionally, Go breaks on the first invalid executing message per block rather than checking all messages before returning a verdict.

Preconditions

- [?] `Valid()` — **Uncertain** — No in-scope callers are available; it cannot be determined from the code whether the Interop struct invariants (logsDBs.Keys == CHAIN_IDS, verifiedDB.Valid(), AllLogsDBsConsistentWithChainData, chains.Keys == CHAIN_IDS, etc.) always hold at every call site.
- [?] `blocksAtTS.Keys == CHAIN_IDS` — **Uncertain** — No in-scope callers are available; it cannot be determined whether the blocksAtTimestamp argument always covers exactly CHAIN_IDS at every call site.
- [?] `AdvancesAllLogsDBs(ts, blocksAtTS)` — **Uncertain** — No in-scope callers are available; it cannot be determined whether each chain's logsDB is at the correct tip (LatestSealedBlock().number <= blocksAtTS[chainID].number <= LatestSealedBlock().number + 1, or None and ts == activationTimestamp) before every call.
- [?] `IsCorrectFrontierView(view, blocksAtTS)` — **Uncertain** — No in-scope callers are available; it cannot be determined whether the frontier view always carries view.BlockInfo(chainID) == chains[chainID].BlockInfo(blocksAtTS[chainID]).value and matching BlockLogs for every chain in CHAIN_IDS at call time.

Postconditions

- [?] `Valid()` — **Uncertain** — i.newInvalidHead is invoked in three branches (first-block hash-mismatch, block hash-mismatch, and invalid-exec-msgs) but has no Dafny spec; its frame safety is not established. All other called functions (l1Inclusion, db.OpenBlock, db.FirstSealedBlock, verifyExecutingMessage) carry 'modifies nothing' in their specs and cannot disturb Valid(). If newInvalidHead writes to verifiedDB, logsDBs, or chains, the invariants constituting Valid() could be broken.
- [x] `result.timestamp == ts` — **Satisfied**
- [x] `result.l2Heads == blocksAtTS` — **Satisfied**

- `|result.invalidHeads| == 0 ==> forall chainID in CHAIN_IDS :: BlockIsCrossValid(view.BlockInfo(chainID).timestam` — **Violated** — When view.block(chainID) returns false for some chain C and db.OpenBlock(expectedBlock.Number) returns ErrSkipped because C's block is the first sealed entry in its logsDB, the code calls db.FirstSealedBlock(), confirms the block number matches expectedBlock.Number, and executes 'continue' without iterating over execMsgs at all. If the hash also matches, no entry is added to result.InvalidHeads, so |result.invalidHeads| == 0. But any executing messages that exist in that first block are never passed to verifyExecutingMessage, so neither ValidExecutingMessage nor InitMsgPresent is established for them. BlockIsCrossValid (which quantifies over all execMsgs in chains[C].BlockLogs(blocksAtTS[C]).value) and AllInitMsgsPresent may therefore be false, violating the postcondition's consequent. The Dafny LogsDB axiom |BlockLogs(FirstSealedBlock().value.number).execMsgs| == 0 is a model assumption not enforced on the concrete Go logsDB, so the violation is reachable.

Violation Scenarios

Precondition `Valid()` (uncertain):

No in-scope callers are available; it cannot be determined from the code whether the Interop struct invariants (logsDBs.Keys == CHAIN_IDS, verifiedDB.Valid(), AllLogsDBsConsistentWithChainData, chains.Keys == CHAIN_IDS, etc.) always hold at every call site.

Precondition `blocksAtTS.Keys == CHAIN_IDS` (uncertain):

No in-scope callers are available; it cannot be determined whether the blocksAtTimestamp argument always covers exactly CHAIN_IDS at every call site.

Precondition `AdvancesAllLogsDBs(ts, blocksAtTS)` (uncertain):

No in-scope callers are available; it cannot be determined whether each chain's logsDB is at the correct tip (LatestSealedBlock().number <= blocksAtTS[chainID].number <= LatestSealedBlock().number + 1, or None and ts == activationTimestamp) before every call.

Precondition `IsCorrectFrontierView(view, blocksAtTS)` (uncertain):

No in-scope callers are available; it cannot be determined whether the frontier view always carries view.BlockInfo(chainID) == chains[chainID].BlockInfo(blocksAtTS[chainID]).value and matching BlockLogs for every chain in CHAIN_IDS at call time.

Postcondition `Valid()` (uncertain):

i.newInvalidHead is invoked in three branches (first-block hash-mismatch, block hash-mismatch, and invalid-exec-msgs) but has no Dafny spec; its frame safety is not established. All other called functions (l1Inclusion, db.OpenBlock, db.FirstSealedBlock, verifyExecutingMessage) carry 'modifies nothing' in their specs and cannot disturb Valid(). If newInvalidHead writes to verifiedDB, logsDBs, or chains, the invariants constituting Valid() could be broken.

Postcondition `|result.invalidHeads| == 0 ==> forall chainID in CHAIN_IDS :: BlockIsCrossValid(view.BlockInfo(chainID).timestamp, c` (violated):

When view.block(chainID) returns false for some chain C and db.OpenBlock(expectedBlock.Number) returns ErrSkipped because C's block is the first sealed entry in its logsDB, the code calls db.FirstSealedBlock(), confirms the block number matches expectedBlock.Number, and executes 'continue' without iterating over execMsgs at all. If the hash also matches, no entry is added to result.InvalidHeads, so |result.invalidHeads| == 0. But any executing messages that exist in that first block are never passed to verifyExecutingMessage, so neither ValidExecutingMessage nor InitMsgPresent is established for them. BlockIsCrossValid (which quantifies over all execMsgs in chains[C].BlockLogs(blocksAtTS[C]).value) and AllInitMsgsPresent may therefore be false, violating the postcondition's consequent. The Dafny LogsDB axiom |BlockLogs(FirstSealedBlock().value.number).execMsgs| == 0 is a model assumption not enforced on the concrete Go logsDB, so the violation is reachable.

---

11. `applyRewindPlan` → `Interop.ApplyRewindPlan`

Go file: `supernode/activity/interop/interop.go:838`
Dafny: `dafny-models/Interop.dfy` — class `Interop`, method `ApplyRewindPlan`
Notes: In Go, resetChainEnginesIfNeeded is called only after all logsDB rewinds succeed; the Dafny model calls RewindChainEngines before the logsDB steps and treats engine-rewind success as independent of logsDB outcomes. If any logsDB operation fails in Go, the engine reset is skipped entirely, making the rewind sequence non-idempotent under partial failure.

Preconditions

- ☑ `Valid()` — **Satisfied**
- ☑ `AllLogsDBsConsistentWithChainData()` — **Satisfied**
- ☑ `verifiedDB.pendingTransition.Some?` — **Satisfied**
- ☑ `verifiedDB.pendingTransition.value.decision == Rewind` — **Satisfied**
- ☑ `verifiedDB.pendingTransition.value.rewind == Some(plan)` — **Satisfied**
- ☑ `ValidRewindPlan(plan)` — **Satisfied**
- ☑ `PlanConsistentWithVerified(plan)` — **Satisfied**
- ☑ `plan.resetAllChainsTo.Some? ==> PlanConsistentWithAllLogs(plan)` — **Satisfied**
- ☑ `plan.resetAllChainsTo.Some? ==> AllDBsInSyncUpTo(plan.resetAllChainsTo.value)` — **Satisfied**
- ☑ `AllVerifiedCrossValid()` — **Satisfied**

Postconditions

- [?] `Valid()` — **Uncertain** — If LogsDB.Clear() or LogsDB.Rewind() fails in Go and leaves the logsDB in a partially-modified state, AllLogsDBsConsistentWithChainData() — a component of Valid() — may not hold. The Dafny LogsDB model never fails; the Go implementation can. All other components of Valid() (verifiedDB.Valid() via VerifiedDB.Rewind's ensures, VerifiedHeadsAreHighestBlocksUpToTimestamp() because remaining logsDB blocks at n > old verifiedHead for any remaining ts have timestamps >= rewindAtOrAfter > ts, AllVerifiedHeadsBoundedByTimestamp() because l2Heads and BlockInfo are unchanged) are maintained even under partial failure.
- [?] `AllLogsDBsConsistentWithChainData()` — **Uncertain** — LogsDB.Clear() and LogsDB.Rewind() are modeled in Dafny as always-succeeding; their ensures clauses then guarantee consistency. In Go both can return errors. If a failed Clear() or Rewind() atomically leaves the logsDB unchanged, the precondition guarantees AllLogsDBsConsistentWithChainData() still holds. If a failed operation partially modifies the logsDB's internal index in a way that makes FindSealedBlock return data inconsistent with chain data, the predicate can be violated. This cannot be determined from the Go source alone.
- ☑ `ValidRewindPlan(plan)` — **Satisfied**
- ☑ `verifiedDB.pendingTransition == old(verifiedDB.pendingTransition)` — **Satisfied**
- ☑ `PlanConsistentWithVerified(plan)` — **Satisfied**
- ☑ `plan.resetAllChainsTo.Some? ==> PlanConsistentWithAllLogs(plan)` — **Satisfied**
- ☑ `plan.resetAllChainsTo.Some? && success ==> RewoundAllLogsDB(plan)` — **Satisfied**
- ☑ `plan.resetAllChainsTo.Some? ==> AllDBsInSyncUpTo(plan.resetAllChainsTo.value)` — **Satisfied**
- ☑ `success ==> AllDBsInSync()` — **Satisfied**
- ☑ `AllVerifiedCrossValid()` — **Satisfied**

Callsite Analysis

Caller	Precondition Verdicts
`applyPendingTransition` (supernode/activity/interop/interop.go:682)	`Valid()` ✓ — applyPendingTransition's Dafny spec lists Valid() as a requires clause, so Valid() is established before applyRewindPlan is called.; `AllLogsDBsConsistentWithChainData()` ✓ — applyPendingTransition's Dafny spec explicitly requires AllLogsDBsConsistentWithChainData(), so it holds at the call site.; `verifiedDB.pendingTransition.Some?` ✓ — applyPendingTransition's spec requires verifiedDB.GetPendingTransition() == Some(pending), guaranteeing pendingTransition.Some? before the call.; `verifiedDB.pendingTransition.value.decision == Rewind` ✓ — applyRewindPlan is only reached inside case DecisionRewind at line 682, so the decision is Rewind at every call site.; `verifiedDB.pendingTransition.value.rewind == Some(plan)` ✓ — ValidPendingTransition (implied by Valid()) guarantees rewind.Some? when decision == Rewind. The call passes *pending.Rewind, so plan equals pending.rewind.value. The nil guard at line ~671 ensures pending.Rewind != nil before the dereference.; `ValidRewindPlan(plan)` ✓ — ValidPendingTransition (implied by Valid() + verifiedDB.pendingTransition.Some?) requires ValidRewindPlan(pending.rewind.value). plan == pending.rewind.value, so this holds.; `PlanConsistentWithVerified(plan)` ✓ — applyPendingTransition's spec requires PendingTransitionIsConsistent(). For the Rewind decision, PendingTransitionIsConsistent() unfolds to TransitionConsistentWithVerified(pending), which for Rewind equals PlanConsistentWithVerified(pending.rewind.value) == PlanConsistentWithVerified(plan).; `plan.resetAllChainsTo.Some? ==> PlanConsistentWithAllLogs(plan)` ✓ — PendingTransitionIsConsistent() for the Rewind case also expands to TransitionConsistentWithLogs(pending), which for Rewind reads: forall k in logsDBs.Keys && resetAllChainsTo.Some? ==> k in targetHeads && PlanConsistentWithLogs(plan, k). This is exactly PlanConsistentWithAllLogs(plan) under the Some? antecedent.; `plan.resetAllChainsTo.Some? ==> AllDBsInSyncUpTo(plan.resetAllChainsTo.value)` ✓ — PendingTransitionIsConsistent() for the Rewind case explicitly includes: p.rewind.value.resetAllChainsTo.Some? ==> AllDBsInSyncUpTo(p.rewind.value.resetAllChainsTo.value). This is exactly the precondition.; `AllVerifiedCrossValid()` ✓ — applyPendingTransition's Dafny spec requires AllVerifiedCrossValid() before the call.

Violation Scenarios

Postcondition `Valid()` (uncertain):

If LogsDB.Clear() or LogsDB.Rewind() fails in Go and leaves the logsDB in a partially-modified state, AllLogsDBsConsistentWithChainData() — a component of Valid() — may not hold. The Dafny LogsDB model never fails; the Go implementation can. All other components of Valid() (verifiedDB.Valid() via VerifiedDB.Rewind's ensures, VerifiedHeadsAreHighestBlocksUpToTimestamp() because remaining logsDB blocks at n > old verifiedHead for any remaining ts have timestamps >= rewindAtOrAfter > ts, AllVerifiedHeadsBoundedByTimestamp() because l2Heads and BlockInfo are unchanged) are maintained even under partial failure.

Postcondition `AllLogsDBsConsistentWithChainData()` (uncertain):

LogsDB.Clear() and LogsDB.Rewind() are modeled in Dafny as always-succeeding; their ensures clauses then guarantee consistency. In Go both can return errors. If a failed Clear() or Rewind() atomically leaves the logsDB unchanged, the precondition guarantees AllLogsDBsConsistentWithChainData() still holds. If a failed operation partially modifies the logsDB's internal index in a way that makes FindSealedBlock return data inconsistent with chain data, the predicate can be violated. This cannot be determined from the Go source alone.

---

12. `verify` → `Interop.Verify`

Go file: `supernode/activity/interop/interop.go:600`
Dafny: `dafny-models/Interop.dfy` — class `Interop`, method `Verify`
Notes: In Go, `verify` delegates to injected closures (`verifyFn` defaulting to `verifyInteropMessages` , and `cycleVerifyFn`) rather than calling those functions directly; this indirection supports test overrides. The Dafny model inlines the three sub-operations (ResolveFrontierVerificationView, VerifyMessages, VerifyCycles) directly. Invalid heads from the cycle check are merged into the message-verification result in both models.

Preconditions

- ☑ `Valid()` — **Satisfied**
- [?] `blocksAtTS.Keys == CHAIN_IDS` — **Uncertain** — If checkPreconditions returns nil while obs.chainsReady is false (e.g., a code path that bypasses the chainsReady check), obs.BlocksAtTS.Keys would not equal CHAIN_IDS, violating this precondition. Not provable safe from the listed Dafny specs alone.
- ☑ `BlocksExistedOnChain(blocksAtTS)` — **Satisfied**
- [?] `AdvancesAllLogsDBs(ts, blocksAtTS)` — **Uncertain** — If checkPreconditions returns nil when obs.l2sConsistent or obs.l1Consistent is false (while obs.chainsReady is true), AdvancesAllLogsDBs(obs.nextTimestamp, obs.blocksAtTS) is not guaranteed by any listed spec, violating this precondition.

Postconditions

- ☑ `Valid()` — **Satisfied**
- ☑ `result.timestamp == ts` — **Satisfied**
- ☑ `result.l2Heads == blocksAtTS` — **Satisfied**
- ☑ `|result.invalidHeads| == 0 ==> ResultIsCrossValid(result)` — **Satisfied**

Callsite Analysis

Caller	Precondition Verdicts
`progressInterop` (supernode/activity/interop/interop.go:522)	`Valid()` ✓ — ProgressInterop (Dafny) carries Valid() as a precondition. observeRound has `requires Valid()` and `ensures Valid()` (modifies only chains.Values). checkPreconditions is not in the Dafny spec list, but from the Go code it accepts obs by value and returns a *StepOutput without touching Interop fields. Therefore Valid() is unaffected between the observeRound return and the verify call at line 522.; `blocksAtTS.Keys == CHAIN_IDS` ? — obs.BlocksAtTS is passed as blocksAtTS. observeRound ensures ValidRoundObservation(obs) (`obs.chainsReady ==> obs.blocksAtTS.Keys == CHAIN_IDS`) and ObservationConsistentWithChainState(obs) (`0 <

Violation Scenarios

Precondition **`blocksAtTS.Keys == CHAIN_IDS`** (uncertain):

If checkPreconditions returns nil while obs.chainsReady is false (e.g., a code path that bypasses the chainsReady check), obs.BlocksAtTS.Keys would not equal CHAIN_IDS, violating this precondition. Not provable safe from the listed Dafny specs alone.

Precondition **`AdvancesAllLogsDBs(ts, blocksAtTS)`** (uncertain):

If checkPreconditions returns nil when obs.l2sConsistent or obs.l1Consistent is false (while obs.chainsReady is true), AdvancesAllLogsDBs(obs.nextTimestamp, obs.blocksAtTS) is not guaranteed by any listed spec, violating this precondition.

---

13. `observeRound` → `Interop.ObserveRound`

Go file: `supernode/activity/interop/interop.go:532`
Dafny: `dafny-models/Interop.dfy` — class `Interop`, method `ObserveRound`
Notes: Go uses a two-phase L1 consistency check: it tracks both L1NeedsRewind (which triggers a rewind from the last-verified L1) and l1Consistent (whether all chains agree on L1). The Dafny model uses a single l1Consistent flag and does not model L1NeedsRewind as a distinct concept.

Preconditions

- ☑ `Valid()` — **Satisfied**
- ☑ `AllDBsInSync()` — **Satisfied**

Postconditions

- ☑ `Valid()` — **Satisfied**
- ☑ `AllDBsInSync()` — **Satisfied**
- ☐ `ValidRoundObservation(obs)` — **Violated** — ValidRoundObservation requires (!obs.l1Consistent ==> obs.lastVerifiedTS.Some?). When verifiedDB is empty (initialized=false on the LastTimestamp call), obs.LastVerifiedTS stays nil and obs.LastVerified stays nil. If checkChainsReady then returns successfully (obs.ChainsReady=true, obs.BlocksAtTS and obs.L1Heads set), the guard 'if obs.LastVerified != nil' is false, so the L1NeedsRewind branch is skipped entirely. Execution falls through to the frontier-heads SameL1Chain call and sets obs.L1Consistent = same. If same=false, obs.L1Consistent=false while obs.LastVerifiedTS=nil, violating the implication. The same violation occurs on the NotFound early-return path: obs.L1Consistent defaults to false (Go zero value) while obs.LastVerifiedTS may be nil when the verifiedDB is uninitialized.
- ☐ `ObservationConsistentWithVerified(obs)` — **Violated** — ObservationConsistentWithVerified has ValidRoundObservation(obs) as a predicate precondition. The same scenario that violates POST3 (empty verifiedDB + frontier L1 check returns false) makes ValidRoundObservation(obs) false, so calling ObservationConsistentWithVerified(obs) is ill-typed and

the ensures clause is violated. Additionally, the sub-condition obs.nextTimestamp == NextTimestamp() in the uninitialized branch (initialized=false) depends on firstVerifiableTimestamp() returning exactly activationTimestamp; this function carries no Dafny spec, so conformance cannot be confirmed from specs alone. In the initialized branch and under a valid ValidRoundObservation, the remaining sub-conditions hold: obs.LastVerifiedTS matches verifiedDB.lastTimestamp by direct assignment; the rewind sub-condition holds because Valid()/Sequential and AllDBsInSync() guarantee lastTS-1 is in verifiedDB.db when activationTimestamp < lastTS; and the AdvancesVerifiedDB sub-condition is vacuously true because obs.L2sConsistent is never set to true in observeRound (Go zero value is false).

- ☐ `ObservationConsistentWithLogs(obs)` — **Violated** — ObservationConsistentWithLogs has both ValidRoundObservation(obs) and ObservationConsistentWithVerified(obs) as predicate preconditions. The POST3-violation scenario (empty verifiedDB + frontier L1 check returns false) makes ValidRoundObservation(obs) false, so the ensures clause is violated. In scenarios where ValidRoundObservation does hold: the rewind sub-condition (!obs.l1Consistent && lastVerifiedTS.value > activationTimestamp ==> sealed blocks for lastTS-1 exist) is satisfied because AllDBsInSync() guarantees DBsInSyncUpTo through lastTS, which covers lastTS-1; the advance sub-condition (chainsReady && l2sConsistent && l1Consistent ==> AdvancesAllLogsDBs) is vacuously true because observeRound never sets obs.L2sConsistent=true (Go zero value is false, a structural divergence from the Dafny model).
- ☑ `ObservationConsistentWithChainState(obs)` — **Satisfied**

Callsite Analysis

Caller	Precondition Verdicts
`progressInterop` (supernode/activity/interop/interop.go:513)	`Valid()` ✓ — The Dafny spec for progressInterop lists Valid() as a requires clause. observeRound is the first call in progressInterop (line 513), so Valid() is established at the call site with no intervening mutations.; `AllDBsInSync()` ✓ — The Dafny spec for progressInterop lists AllDBsInSync() as a requires clause. observeRound is the first call in progressInterop (line 513), so AllDBsInSync() is established at the call site with no intervening mutations.

Violation Scenarios

Postcondition `ValidRoundObservation(obs)` (violated):

ValidRoundObservation requires (!obs.l1Consistent ==> obs.lastVerifiedTS.Some?). When verifiedDB is empty (initialized=false on the LastTimestamp call), obs.LastVerifiedTS stays nil and obs.LastVerified stays nil. If checkChainsReady then returns successfully (obs.ChainsReady=true, obs.BlocksAtTS and obs.L1Heads set), the guard 'if obs.LastVerified != nil' is false, so the L1NeedsRewind branch is skipped entirely. Execution falls through to the frontier-heads SameL1Chain call and sets obs.L1Consistent = same. If same=false, obs.L1Consistent=false while obs.LastVerifiedTS=nil, violating the implication. The same violation occurs on the NotFound early-return path: obs.L1Consistent defaults to false (Go zero value) while obs.LastVerifiedTS may be nil when the verifiedDB is uninitialized.

Postcondition `ObservationConsistentWithVerified(obs)` (violated):

ObservationConsistentWithVerified has ValidRoundObservation(obs) as a predicate precondition. The same scenario that violates POST3 (empty verifiedDB + frontier L1 check returns false) makes ValidRoundObservation(obs) false, so calling ObservationConsistentWithVerified(obs) is ill-typed and the ensures clause is violated. Additionally, the sub-condition obs.nextTimestamp == NextTimestamp() in the uninitialized branch (initialized=false) depends on firstVerifiableTimestamp() returning exactly activationTimestamp; this function carries no Dafny spec, so conformance cannot be confirmed from specs alone. In the initialized branch and under a valid ValidRoundObservation, the remaining sub-conditions hold: obs.LastVerifiedTS matches verifiedDB.lastTimestamp by direct assignment; the rewind sub-condition holds because Valid()/Sequential and AllDBsInSync() guarantee lastTS-1 is in verifiedDB.db when activationTimestamp < lastTS; and the AdvancesVerifiedDB sub-condition is vacuously true because obs.L2sConsistent is never set to true in observeRound (Go zero value is false).

Postcondition `ObservationConsistentWithLogs(obs)` (violated):

ObservationConsistentWithLogs has both ValidRoundObservation(obs) and ObservationConsistentWithVerified(obs) as predicate preconditions. The POST3-violation scenario (empty verifiedDB + frontier L1 check returns false) makes ValidRoundObservation(obs) false, so the ensures clause is violated. In scenarios where ValidRoundObservation does hold: the rewind sub-condition

(!obs.l1Consistent && lastVerifiedTS.value > activationTimestamp ==> sealed blocks for lastTS-1 exist) is satisfied because AllDBsInSync() guarantees DBsInSyncUpTo through lastTS, which covers lastTS-1; the advance sub-condition (chainsReady && l2sConsistent && l1Consistent ==> AdvancesAllLogsDBs) is vacuously true because observeRound never sets obs.L2sConsistent=true (Go zero value is false, a structural divergence from the Dafny model).

14. `applyPendingTransition` → `Interop.ApplyPendingTransition`

Go file: `supernode/activity/interop/interop.go:673`
Dafny: `dafny-models/Interop.dfy` — class `Interop`, method `ApplyPendingTransition`
Notes: Before any invalidateBlock call, Go pauses all chains' validation nodes via PauseAndStopVN and resumes non-invalidated chains afterwards; this VN lifecycle management has no counterpart in the Dafny model. Go also has a nil-result guard that calls ClearPendingTransition and returns gracefully when pending.Result == nil; the Dafny model treats this scenario as a precondition violation rather than a recoverable runtime condition.

Preconditions

- ☑ `Valid()` — **Satisfied**
- ☑ `verifiedDB.GetPendingTransition() == Some(pending)` — **Satisfied**
- ☑ `PendingTransitionIsConsistent()` — **Satisfied**
- ☑ `TransitionIsCrossValid(pending)` — **Satisfied**
- ☑ `AllVerifiedCrossValid()` — **Satisfied**
- ☑ `AllLogsDBsConsistentWithChainData()` — **Satisfied**

Postconditions

- [?] `Valid()` — **Uncertain** — DecisionInvalidate path: invalidateBlock has no Dafny spec, so it is unknown whether it preserves the Valid() invariants on verifiedDB and logsDBs. ClearPendingTransition only modifies verifiedDB.pendingTransition and cannot repair deeper invariant violations. DecisionAdvance path: commitVerifiedResult has no Dafny spec; if it leaves verifiedDB.db or logsDBs in a state that violates Valid() (e.g. non-sequential timestamps or mismatched l2Heads.Keys), Valid() would not hold despite the subsequent ClearPendingTransition call.
- [?] `PendingTransitionIsConsistent()` — **Uncertain** — DecisionInvalidate success path: after ClearPendingTransition sets pendingTransition == None, PendingTransitionIsConsistent() requires AllDBsInSync(). invalidateBlock has no Dafny spec guaranteeing AllDBsInSync() is preserved. DecisionAdvance success path: after commitVerifiedResult and ClearPendingTransition, AllDBsInSync() is required; commitVerifiedResult has no spec confirming it performs the VerifiedDB.Commit step that would bring verifiedDB.db in line with the already-updated logsDBs. DecisionInvalidate error path with failedAny: pendingTransition == Some(pending) remains but the Invalidate extra condition AllDBsInSync() may be broken by partial invalidateBlock calls.
- [?] `AllVerifiedCrossValid()` — **Uncertain** — DecisionInvalidate path: invalidateBlock has no Dafny spec; if it modifies verifiedDB.db in a way that adds or alters entries, previously cross-valid results might no longer satisfy ResultIsCrossValid under the new chain state. DecisionAdvance path: commitVerifiedResult has no Dafny spec; if it does not perform the VerifiedDB.Commit that establishes AllVerifiedCrossValid() for the new timestamp, the postcondition is unverified.
- [?] `AllLogsDBsConsistentWithChainData()` — **Uncertain** — AllLogsDBsConsistentWithChainData() is a conjunct of Valid(). Same uncertainty applies: invalidateBlock (DecisionInvalidate) and commitVerifiedResult (DecisionAdvance) have no Dafny specs confirming they preserve the LogsDB-to-chain-data consistency invariant. ClearPendingTransition does not repair logsDB state.
- [?] `madeProgress.None? ==> verifiedDB.pendingTransition == Some(pending)` — **Uncertain** — DecisionAdvance error from commitVerifiedResult: commitVerifiedResult has no Dafny spec. It is unknown whether it modifies verifiedDB.pendingTransition before returning an error. If it does (e.g. if it internally calls ClearPendingTransition or sets pendingTransition to None as part of a partial commit), the error return would have pendingTransition != Some(pending), violating this postcondition. All other error paths

(applyRewindPlan failure, persistFrontierLogs failure, failedAny in Invalidate) do not call ClearPendingTransition first and the relevant modifies clauses (applyRewindPlan: verifiedDB.pendingTransition == old(verifiedDB.pendingTransition); ChainContainer.InvalidateBlock: modifies this only; persistFrontierLogs: modifies logsDBs.Values and chains.Values only) confirm pendingTransition is unchanged in those cases.

- [?] `madeProgress.Some? ==> verifiedDB.pendingTransition == None` — **Uncertain** — All successful Go return paths call ClearPendingTransition last, which ensures pendingTransition == None — but ClearPendingTransition requires Valid() as a precondition. For DecisionInvalidate: Valid() after invalidateBlock calls is not guaranteed (no spec for invalidateBlock), so the precondition of ClearPendingTransition may be violated and its postcondition (pendingTransition == None) cannot be relied upon. For DecisionAdvance: Valid() after commitVerifiedResult is not guaranteed (no spec), same issue.
- ☑ `madeProgress.Some? ==> (madeProgress.value <==> pending.decision == Advance)` — **Satisfied**
- [?] `madeProgress.Some? ==> AllDBsInSync()` — **Uncertain** — DecisionInvalidate success: after invalidateBlock calls, AllDBsInSync() requires each chain's logsDB LatestSealedBlock to match the verifiedDB's latest verified l2Head. invalidateBlock has no Dafny spec guaranteeing this; it may rewind some logsDBs but leave verifiedDB.db unchanged, or vice versa. DecisionAdvance success: persistFrontierLogs postcondition ensures UpdatedAllLogsDBs (logsDBs advanced), but commitVerifiedResult has no Dafny spec to confirm it performs the VerifiedDB.Commit step that would make verifiedDB.db consistent with those logsDB updates, so AllDBsInSync() is unverified.
- [?] `madeProgress.None? ==> TransitionIsCrossValid(pending)` — **Uncertain** — TransitionIsCrossValid is trivially true for Rewind and Invalidate decisions, so error paths in those branches satisfy this postcondition trivially. For DecisionAdvance errors from commitVerifiedResult: commitVerifiedResult has no Dafny spec; if it modifies logsDBs (which AllInitMsgsPresent/InitMsgInLogsDB depends on) in a way that removes entries needed by ResultIsCrossValid(pending.result.value), the postcondition could be violated. persistFrontierLogs errors are safe because its partial-update postcondition only adds blocks to logsDBs (Contains is monotone), which can only maintain or strengthen InitMsgInLogsDB queries.

Callsite Analysis

Caller	Precondition Verdicts
`progressAndRecord` (supernode/activity/interop/interop.go:467)	`Valid()` ✓ — Valid() is a direct precondition of progressAndRecord. GetPendingTransition() is frame-safe (no modifies clause), so Valid() is unmodified between progressAndRecord entry and the call.; `verifiedDB.GetPendingTransition() == Some(pending)` ✓ — pending is the non-nil result of GetPendingTransition(); applyPendingTransition receives *pending. Nothing between the GetPendingTransition call and the applyPendingTransition call modifies verifiedDB.pendingTransition, so GetPendingTransition() == Some(*pending) at the call.; `PendingTransitionIsConsistent()` ✓ — PendingTransitionIsConsistent() is a direct precondition of progressAndRecord; GetPendingTransition() is frame-safe, so it holds unchanged at the call.; `TransitionIsCrossValid(pending)` ✓ — progressAndRecord precondition: GetPendingTransition().Some? ==> TransitionIsCrossValid(...). Since pending != nil the Some? branch fires, yielding TransitionIsCrossValid(*pending).; `AllVerifiedCrossValid()` ✓ — AllVerifiedCrossValid() is a direct precondition of progressAndRecord; frame-safe GetPendingTransition() does not change verifiedDB.db.; `AllLogsDBsConsistentWithChainData()` ✓ — AllLogsDBsConsistentWithChainData() is a conjunct of Valid(). Valid() is a direct precondition of progressAndRecord; frame-safe GetPendingTransition() does not disturb it.
`progressAndRecord` (supernode/activity/interop/interop.go:490)	`Valid()` ✓ — progressInterop postcondition ensures Valid(). buildPendingTransition is frame-safe. SetPendingTransition postcondition ensures Valid(). All three preserve Valid() up to the call.; `verifiedDB.GetPendingTransition() == Some(pending)` ✓ — SetPendingTransition postcondition ensures pendingTransition == Some(pendingTx). applyPendingTransition(pendingTx) is called immediately after with no intervening state changes, so GetPendingTransition() == Some(pendingTx) == Some(pending).; `PendingTransitionIsConsistent()` ✓ — progressInterop postcondition yields AllDBsInSync(). buildPendingTransition postcondition yields TransitionConsistentWithVerified, TransitionConsistentWithLogs, and TransitionConsistentWithChainState (via OutputConsistentWithChainState from progressInterop). SetPendingTransition does not change verifiedDB.db or logsDBs. For each decision: Invalidate/Advance require AllDBsInSync() (preserved through frame-safe operations); Rewind extra condition AllDBsInSyncUpTo(resetAllChainsTo.value) follows because that timestamp <= LastTimestamp() and AllDBsInSync() implies AllDBsInSyncUpTo for any timestamp at or below LastTimestamp().; `TransitionIsCrossValid(pending)` ✓ — progressInterop postcondition: output.AdvanceOutput? ==> ResultIsCrossValid(output.result). buildPendingTransition postcondition: output.AdvanceOutput? <==> pendingTx.decision.Advance? and pendingTx.result.value == output.result for Advance. TransitionIsCrossValid is trivially true for Rewind and Invalidate decisions.; `AllVerifiedCrossValid()` ✓ — progressInterop postcondition ensures AllVerifiedCrossValid(). buildPendingTransition is frame-safe. SetPendingTransition does not modify verifiedDB.db (postcondition: db == old(db)), so AllVerifiedCrossValid() is preserved.; `AllLogsDBsConsistentWithChainData()` ✓ — progressInterop postcondition ensures Valid() which includes AllLogsDBsConsistentWithChainData(). buildPendingTransition is frame-safe; SetPendingTransition only modifies verifiedDB.pendingTransition. Condition preserved at the call.

Violation Scenarios

Postcondition `Valid()` (uncertain):

DecisionInvalidate path: invalidateBlock has no Dafny spec, so it is unknown whether it preserves the Valid() invariants on verifiedDB and logsDBs. ClearPendingTransition only modifies verifiedDB.pendingTransition and cannot repair deeper invariant violations. DecisionAdvance path: commitVerifiedResult has no Dafny spec; if it leaves verifiedDB.db or logsDBs in a state that violates Valid() (e.g. non-sequential timestamps or mismatched l2Heads.Keys), Valid() would not hold despite the subsequent ClearPendingTransition call.

Postcondition `PendingTransitionIsConsistent()` (uncertain):

DecisionInvalidate success path: after ClearPendingTransition sets pendingTransition == None, PendingTransitionIsConsistent() requires AllDBsInSync(). invalidateBlock has no Dafny spec guaranteeing AllDBsInSync() is preserved. DecisionAdvance success path: after commitVerifiedResult and ClearPendingTransition, AllDBsInSync() is required; commitVerifiedResult has no spec confirming it performs the VerifiedDB.Commit step that would bring verifiedDB.db in line with the already-updated logsDBs. DecisionInvalidate error path with failedAny: pendingTransition == Some(pending) remains but the Invalidate extra condition AllDBsInSync() may be broken by partial invalidateBlock calls.

Postcondition `AllVerifiedCrossValid()` (uncertain):

DecisionInvalidate path: invalidateBlock has no Dafny spec; if it modifies verifiedDB.db in a way that adds or alters entries, previously cross-valid results might no longer satisfy ResultIsCrossValid under the new chain state. DecisionAdvance path: commitVerifiedResult has no Dafny spec; if it does not perform the VerifiedDB.Commit that establishes AllVerifiedCrossValid() for the new timestamp, the postcondition is unverified.

Postcondition `AllLogsDBsConsistentWithChainData()` (uncertain):

AllLogsDBsConsistentWithChainData() is a conjunct of Valid(). Same uncertainty applies: invalidateBlock (DecisionInvalidate) and commitVerifiedResult (DecisionAdvance) have no Dafny specs confirming they preserve the LogsDB-to-chain-data consistency invariant. ClearPendingTransition does not repair logsDB state.

Postcondition `madeProgress.None? ==> verifiedDB.pendingTransition == Some(pending)` (uncertain):

DecisionAdvance error from commitVerifiedResult: commitVerifiedResult has no Dafny spec. It is unknown whether it modifies verifiedDB.pendingTransition before returning an error. If it does (e.g. if it internally calls ClearPendingTransition or sets pendingTransition to None as part of a partial commit), the error return would have pendingTransition != Some(pending), violating this postcondition. All other error paths (applyRewindPlan failure, persistFrontierLogs failure, failedAny in Invalidate) do not call ClearPendingTransition first and the relevant modifies clauses (applyRewindPlan: verifiedDB.pendingTransition == old(verifiedDB.pendingTransition); ChainContainer.InvalidateBlock: modifies this only; persistFrontierLogs: modifies logsDBs.Values and chains.Values only) confirm pendingTransition is unchanged in those cases.

Postcondition `madeProgress.Some? ==> verifiedDB.pendingTransition == None` (uncertain):

All successful Go return paths call ClearPendingTransition last, which ensures pendingTransition == None — but ClearPendingTransition requires Valid() as a precondition. For DecisionInvalidate: Valid() after invalidateBlock calls is not guaranteed (no spec for invalidateBlock), so the precondition of ClearPendingTransition may be violated and its postcondition (pendingTransition == None) cannot be relied upon. For DecisionAdvance: Valid() after commitVerifiedResult is not guaranteed (no spec), same issue.

Postcondition `madeProgress.Some? ==> AllDBsInSync()` (uncertain):

DecisionInvalidate success: after invalidateBlock calls, AllDBsInSync() requires each chain's logsDB LatestSealedBlock to match the verifiedDB's latest verified l2Head. invalidateBlock has no Dafny spec guaranteeing this; it may rewind some logsDBs but leave verifiedDB.db unchanged, or vice versa. DecisionAdvance success: persistFrontierLogs postcondition ensures UpdatedAllLogsDBs (logsDBs advanced), but commitVerifiedResult has no Dafny spec to confirm it performs the VerifiedDB.Commit step that would make verifiedDB.db consistent with those logsDB updates, so AllDBsInSync() is unverified.

Postcondition `madeProgress.None? ==> TransitionIsCrossValid(pending)` (uncertain):

TransitionIsCrossValid is trivially true for Rewind and Invalidate decisions, so error paths in those branches satisfy this postcondition trivially. For DecisionAdvance errors from commitVerifiedResult: commitVerifiedResult has no Dafny spec; if it modifies logsDBs (which AllInitMsgsPresent/InitMsgInLogsDB depends on) in a way that removes entries needed by ResultIsCrossValid(pending.result.value), the postcondition could be violated. persistFrontierLogs errors are safe because its partial-update postcondition only adds blocks to logsDBs (Contains is monotone), which can only maintain or strengthen InitMsgInLogsDB queries.

---

15. `progressInterop` → `Interop.ProgressInterop`

Go file: `supernode/activity/interop/interop.go:512`
Dafny: `dafny-models/Interop.dfy` — class `Interop`, method `ProgressInterop`
Notes: The precondition-check logic is extracted into a separate standalone function checkPreconditions in Go; the Dafny model inlines this logic directly inside ProgressInterop. Go also supports a Paused state (controlled by pauseAtTimestamp) that silently halts verification progress; this concept is absent from the Dafny model.

Preconditions

- ☑ `Valid()` — **Satisfied**
- ☑ `AllDBsInSync()` — **Satisfied**
- ☑ `AllVerifiedCrossValid()` — **Satisfied**

Postconditions

- ☑ `Valid()` — **Satisfied**
- ☑ `AllDBsInSync()` — **Satisfied**
- ☑ `AllVerifiedCrossValid()` — **Satisfied**
- ☑ `ValidStepOutput(output, obs)` — **Satisfied**
- [?] `OutputConsistentWithVerified(output, obs)` — **Uncertain** — For the AdvanceOutput case, OutputConsistentWithVerified requires AdvancesVerifiedDB(result.timestamp, result.l2Heads). ObservationConsistentWithVerified (ensured by observeRound) only guarantees AdvancesVerifiedDB when obs.chainsReady && obs.l2sConsistent && obs.l1Consistent. However, observeRound in Go never sets obs.L2sConsistent — it retains its zero value (false) — so the antecedent of that implication is never satisfied and AdvancesVerifiedDB is not established through any listed Dafny postcondition. An informal argument (AllDBsInSync() precondition + checkChainsReady postcondition AdvancesAllLogsDBs implies AdvancesVerifiedDB) would establish it, but this equivalence is not captured by any Dafny lemma or spec in scope.
- [?] `OutputConsistentWithLogs(output, obs)` — **Uncertain** — OutputConsistentWithLogs carries a requires of OutputConsistentWithVerified(output, obs). For the AdvanceOutput case that requires clause is uncertain (see post 5). For WaitOutput and RewindOutput the requires is satisfied and ObservationConsistentWithLogs (ensured by observeRound) supplies the needed sealed-block property for the RewindOutput path; for AdvanceOutput, checkChainsReady ensures AdvancesAllLogsDBs independently, but since the requires (post 5) is uncertain the overall status of post 6 is uncertain.
- ☑ `OutputConsistentWithChainState(output, obs)` — **Satisfied**
- ☑ `output.AdvanceOutput? ==> ResultIsCrossValid(output.result)` — **Satisfied**

Callsite Analysis

Caller	Precondition Verdicts
`progressAndRecord` (supernode/activity/interop/interop.go:471)	`Valid()` ✓ — progressAndRecord's Dafny spec requires Valid(). Before the call to progressInterop at line 471, only verifiedDB.GetPendingTransition() (frame-safe, no modifies) executes. When pending != nil the function returns early via applyPendingTransition without reaching progressInterop. No state is modified between the entry of progressAndRecord and the call to progressInterop, so Valid() holds at the call site.; `AllDBsInSync()` ✓ — progressAndRecord requires PendingTransitionIsConsistent(). When verifiedDB.GetPendingTransition() returns None (pending == nil in Go), PendingTransitionIsConsistent() reduces to AllDBsInSync(). progressInterop is only reachable when pending == nil (the non-nil branch returns early at line 467). GetPendingTransition() is frame-safe, so AllDBsInSync() is unmodified at the call site.; `AllVerifiedCrossValid()` ✓ — progressAndRecord requires AllVerifiedCrossValid(). Only the frame-safe GetPendingTransition() executes before progressInterop is called, leaving AllVerifiedCrossValid() unmodified at the call site.

Violation Scenarios

Postcondition `OutputConsistentWithVerified(output, obs)` (uncertain):

For the AdvanceOutput case, OutputConsistentWithVerified requires AdvancesVerifiedDB(result.timestamp, result.l2Heads). ObservationConsistentWithVerified (ensured by observeRound) only guarantees AdvancesVerifiedDB when obs.chainsReady && obs.l2sConsistent && obs.l1Consistent. However, observeRound in Go never sets obs.L2sConsistent — it retains its zero value (false)

— so the antecedent of that implication is never satisfied and AdvancesVerifiedDB is not established through any listed Dafny postcondition. An informal argument (AllDBsInSync() precondition + checkChainsReady postcondition AdvancesAllLogsDBs implies AdvancesVerifiedDB) would establish it, but this equivalence is not captured by any Dafny lemma or spec in scope.

Postcondition `OutputConsistentWithLogs(output, obs)` (uncertain):

OutputConsistentWithLogs carries a requires of OutputConsistentWithVerified(output, obs). For the AdvanceOutput case that requires clause is uncertain (see post 5). For WaitOutput and RewindOutput the requires is satisfied and ObservationConsistentWithLogs (ensured by observeRound) supplies the needed sealed-block property for the RewindOutput path; for AdvanceOutput, checkChainsReady ensures AdvancesAllLogsDBs independently, but since the requires (post 5) is uncertain the overall status of post 6 is uncertain.

---

16. `progressAndRecord` → `Interop.ProgressAndRecord`

Go file: `supernode/activity/interop/interop.go:460`
Dafny: `dafny-models/Interop.dfy` — class `Interop`, method `ProgressAndRecord`

Preconditions

- [?] `Valid()` — **Uncertain** — If a prior invocation of `applyPendingTransition` returned a Go error after partially mutating `verifiedDB` or `logsDBs`, `Valid()` (which requires `verifiedDB.Valid()`, `AllLogsDBsConsistentWithChainData()`, sequential commits, etc.) could be broken before the next call to `progressAndRecord`. Dafny models errors as `None` returns and does not cover partial mutations on Go error paths, so this scenario is outside the formal model but observable in the implementation.
- [?] `PendingTransitionIsConsistent()` — **Uncertain** — Same error-path scenario as `Valid()`: if `applyPendingTransition` partially rewinds or advances `verifiedDB` / `logsDBs` before returning a Go error, `logsDBs` and `verifiedDB` could be out of sync, violating `AllDBsInSync()` and therefore `PendingTransitionIsConsistent()` at the next call site.
- [?] `AllVerifiedCrossValid()` — **Uncertain** — If `applyPendingTransition` commits a new timestamp to `verifiedDB` but then fails before updating the logsDBs to match, the new entry in `verifiedDB` would not satisfy `ResultIsCrossValid` (because `AllInitMsgsInLogsDB` could fail), violating `AllVerifiedCrossValid()` at the next entry to `progressAndRecord`.
- [?] `verifiedDB.GetPendingTransition().Some? ==> TransitionIsCrossValid(verifiedDB.GetPendingTransition().value)` — **Uncertain** — If a future version of the code stores a pending transition in `verifiedDB` before the cross-validity of its result is confirmed (e.g., calling `SetPendingTransition` before `progressInterop` validates the blocks), then a crash-and-restart picking up that pending transition would find `TransitionIsCrossValid` not holding. This is not the current code flow, but the condition cannot be proven satisfied purely from the Go code without formal verification.

Postconditions

- ☑ `Valid()` — **Satisfied**
- ☑ `PendingTransitionIsConsistent()` — **Satisfied**
- ☑ `AllVerifiedCrossValid()` — **Satisfied**
- ☑ `verifiedDB.GetPendingTransition().Some? ==> TransitionIsCrossValid(verifiedDB.GetPendingTransition().value)` — **Satisfied**

Callsite Analysis

Caller	Precondition Verdicts
`progress` (supernode/activity/interop/interop.go:386)	`Valid()` ? — `progress` calls `progressAndRecord` as its first statement with no prior action that establishes or checks `Valid()`. `Valid()` must therefore hold on entry to `progress` itself, which requires tracing the full call chain back to the constructor. The constructor ensures `Valid()`, and every in-scope mapped function (including `applyPendingTransition`, `progressInterop`, `refreshCurrentL1OnWait`, `SetPendingTransition`) lists `Valid()` in its postconditions, so the invariant is designed to be maintained across calls. However, Go provides no static enforcement of this; an I/O-error early-exit from `applyPendingTransition` (modifies `this, verifiedDB, chains.Values, logsDBs.Values`) could leave the object in a state where `Valid()` no longer holds before the next invocation of `progress`.; `PendingTransitionIsConsistent()` ? — `progress` performs no action before calling `progressAndRecord`, so `PendingTransitionIsConsistent()` must hold on entry to `progress`. When `verifiedDB.GetPendingTransition() == None`, this collapses to `AllDBsInSync()`; when `Some(p)`, it additionally requires `TransitionConsistentWithVerified(p)`, `TransitionConsistentWithLogs(p)`, and `TransitionConsistentWithChainState(p)`. `applyPendingTransition` ensures `PendingTransitionIsConsistent()` post-call, and `progressInterop` ensures `AllDBsInSync()`, so the invariant is designed to be propagated. The same caveat applies as for `Valid()`: a Go-error path out of `applyPendingTransition` after partial DB mutation could leave `AllDBsInSync()` violated for the next round.; `AllVerifiedCrossValid()` ? — `progress` does not check or establish `AllVerifiedCrossValid()`. The constructor ensures it; `applyPendingTransition` and `progressInterop` both list it in their postconditions; and functions that do not modify `verifiedDB` or `logsDBs.Values` (e.g., `buildPendingTransition`, `refreshCurrentL1OnWait`) preserve it by framing. The same caveat applies: a Go error mid-way through `applyPendingTransition` (which commits to `verifiedDB` before potentially failing) could invalidate a just-committed result, breaking `AllVerifiedCrossValid()` before the next call.; `verifiedDB.GetPendingTransition().Some? ==> TransitionIsCrossValid(verifiedDB.GetPendingTransition().value)` ? — `progress` does not check this condition. The invariant is maintained by: (a) the constructor ensuring it, (b) `applyPendingTransition` postcondition ensuring it when `madeProgress.None?` (pending kept), and clearing the pending when `madeProgress.Some?` (vacuously satisfied). In path 2c of `progressAndRecord`, `TransitionIsCrossValid(pendingTx)` is established from `progressInterop`'s `output.AdvanceOutput? ==> ResultIsCrossValid(output.result)` combined with `buildPendingTransition`'s `pendingTx.decision.Advance? <==> output.AdvanceOutput?` and `pendingTx.decision.Advance? ==> pendingTx.result.value == output.result`. However, if a prior Go error left a pending transition in `verifiedDB` whose cross-validity was not confirmed (e.g., stored before `ResultIsCrossValid` was established), this precondition could be violated.

Violation Scenarios

Precondition `Valid()` (uncertain):

If a prior invocation of `applyPendingTransition` returned a Go error after partially mutating `verifiedDB` or `logsDBs`, `Valid()` (which requires `verifiedDB.Valid()`, `AllLogsDBsConsistentWithChainData()`, sequential commits, etc.) could be broken before the next call to `progressAndRecord`. Dafny models errors as `None` returns and does not cover partial mutations on Go error paths, so this scenario is outside the formal model but observable in the implementation.

Precondition `PendingTransitionIsConsistent()` (uncertain):

Same error-path scenario as `Valid()`: if `applyPendingTransition` partially rewinds or advances `verifiedDB` / `logsDBs` before returning a Go error, `logsDBs` and `verifiedDB` could be out of sync, violating `AllDBsInSync()` and therefore `PendingTransitionIsConsistent()` at the next call site.

Precondition `AllVerifiedCrossValid()` (uncertain):

If `applyPendingTransition` commits a new timestamp to `verifiedDB` but then fails before updating the logsDBs to match, the new entry in `verifiedDB` would not satisfy `ResultIsCrossValid` (because `AllInitMsgsInLogsDB` could fail), violating `AllVerifiedCrossValid()` at the next entry to `progressAndRecord`.

Precondition `verifiedDB.GetPendingTransition().Some? ==> TransitionIsCrossValid(verifiedDB.GetPendingTransition().value)` (uncertain):

If a future version of the code stores a pending transition in `verifiedDB` before the cross-validity of its result is confirmed (e.g., calling `SetPendingTransition` before `progressInterop` validates the blocks), then a crash-and-restart picking up that pending transition would find `TransitionIsCrossValid` not holding. This is not the current code flow, but the condition cannot be proven satisfied purely from the Go code without formal verification.